실전반 5회차 — 낭독 대본

[이론 01] 실전반 5회차

[SLIDE 1] 사람과 협업하고 변경된 기능까지 검증하기

앞 회차에서 만든 에이전트가 사람과 협업하고, 요구가 바뀐 뒤에도 지켜야 할 기준을 유지하도록 만들겠습니다. 사람의 승인과 확인 질문부터 명세에 따른 기능 추가, 시나리오와 회귀 검증까지 연결합니다.

[SLIDE 2] 학습목표

사람의 결정과 정보 확인, AI의 검토 의견을 구별하겠습니다. 변경할 때는 기존 입출력 계약과 금지 조건을 보존하고 수용 기준을 먼저 적습니다. 그 기준을 시나리오로 옮겨 새 기능과 기존 기능을 함께 확인하겠습니다. 마지막에는 제작과 검증의 산출물이 다음 작업에 어떻게 이어지는지 정리합니다.

[SLIDE 3] 목차

사람이 결정하는 자리와 질문으로 정보를 채우는 방법을 배우고 두 실습으로 확인합니다. 이어서 명세와 계약을 배우고 기존 에이전트에 요구사항을 더합니다. 다음은 행동 시나리오와 반복·회귀 검증입니다. 같은 프로젝트를 검증한 뒤 전체 과정을 연결하고 복습과 용어, 여섯 퀴즈를 정리하겠습니다.

[SLIDE 4] HITL: 사람이 결정하는 자리

첫 단원부터 시작하죠. 주제는 사람이 결정하는 자리입니다. 쿨러 사건 기억나시죠? 에이전트가 원인을 가리고, 조치 후보를 비교해서 추천안을 냈습니다. 그런데 그 추천안이 올라온 다음에는 누가 무엇을 결정할까요? 아무도 결정하지 않으면 어떻게 될까요? 여기에 에이전트가 일하는 도중에 사람에게 묻거나 멈추는 때까지 넓혀서, 사람이 끼어드는 자리를 하나씩 찾아보겠습니다.

[SLIDE 5] 추천안이 나온 뒤: 승인과 거절의 갈림

그림 왼쪽부터 볼게요. 에이전트가 있고, 그 옆에 겹쳐 놓은 카드가 추천안이에요. 추천 하나에, 그보다 점수가 낮은 대안이 셋 들어 있었죠. 이 추천안은 실행되지 않은 채로 담당자에게 올라갔습니다.

아래쪽 시간 축을 보세요. 칠해 둔 구간이 승인을 기다린 시간인데, 약 13초입니다. 그 구간이 끝나는 자리에서 운전원이 승인했어요. 담당자 옆에 적힌 승인 시각과, 오른쪽 설비 명령 옆에 적힌 시각을 견줘 보겠습니다. 팬을 최대로 돌리는 설비 명령은 승인 직후에야 만들어졌습니다.

담당자 오른쪽에서는 길이 둘로 갈리죠. 위쪽이 실제로 일어난 승인이에요. 아래쪽은 같은 자리에서 거절했다면 갔을 길입니다.

에이전트는 원인을 가리고 후보를 견주느라, 승인을 기다린 시간보다 훨씬 오래 일했어요. 그런데 설비를 실제로 바꾼 출발점은 그 결과물이 아니었습니다. 그 뒤에 온 사람의 승인이었죠. 그렇다면 왜 굳이 이 자리에 사람이 있어야 할까요?

[SLIDE 6] HITL(Human-in-the-Loop): 자동 처리와 사람의 결정

화면 왼쪽은 값을 조회하고, 원인을 판단하고, 후보를 비교하고, 형식을 검사하는 일입니다. 오른쪽은 그 추천을 실제 조치로 넘길지 결정하는 자리예요. 둘을 가르는 까닭은 틀렸을 때 미치는 영향과 책임이 다르기 때문입니다.

조회나 계산을 다시 해 볼 수는 있어요. 하지만 틀린 조회 결과를 근거로 결정을 내리면 뒤의 조치도 잘못될 수 있습니다. 읽기 도구라도 정보가 노출될 수 있고, 실행 기록이나 비용은 남을 수 있죠. 자동 처리라는 말이 영향이 없다는 뜻은 아닙니다.

팬 속도를 바꾸거나 정비 작업을 지시하면 설비와 업무의 상태가 바뀝니다. 뒤늦게 명령을 취소해도 이미 지난 운전 시간이나 투입한 작업까지 없앨 수는 없어요. 그래서 HYD에서는 추천과 실행 사이에 담당자가 판단할 자리를 둡니다.

HITL, Human-in-the-Loop는 자동화 과정에 사람의 확인이나 결정을 넣는 방식입니다. 사람의 참여는 승인뿐 아니라 빠진 정보를 답하거나 결과를 바로잡는 형태일 수도 있어요. 이 회차에서는 HYD의 승인과 질문, 멈춘 작업의 처리로 그 원리를 보겠습니다.

사람이 버튼을 누른다고 결정의 품질이 저절로 보장되지는 않습니다. 판단할 근거를 볼 수 있어야 하고, 다른 안을 선택하거나 거절할 권한과 시간도 있어야 해요. 자동화가 만든 결과를 사람이 검토할 수 있는 형태로 넘기는 것까지가 이 구조의 역할입니다.

[SLIDE 7] 사람이 개입하는 세 지점

사람이 끼어드는 자리가 승인 하나뿐일까요? 아닙니다. 여기서는 HYD의 대표적인 세 지점을 보겠습니다. 그림 위쪽 띠는 처리의 구간으로, 왼쪽부터 에이전트 단계, 승인 단계, 시스템 실행 순서입니다.

그 아래 세 장면이 사람이 끼어드는 자리예요. 시간 순서대로 왼쪽부터 보겠습니다. 점선을 따라가 보면, 왼쪽 두 장면은 위쪽 띠의 에이전트 단계에 이어져 있죠. 에이전트가 일하는 동안에 생기는 자리예요.

맨 왼쪽은 에이전트의 질문입니다. 필요한 업무 정보가 빠졌을 때 에이전트가 묻고 기다리다가, 담당자가 답하면 이어서 처리해요. 가운데는 멈춘 작업 닫기입니다. 에이전트 작업이 한도를 넘기거나 실행이 실패해서 멈추면, 이건 저절로 끝나지 않아요. 그래서 담당자가 사유를 적고 닫습니다.

맨 오른쪽은 승인 단계에 이어져 있습니다. 추천안이 나온 뒤, 실행하기 전에 승인 역할의 담당자가 승인하거나 거절하는 자리죠. 앞에서 본 그 자리예요.

셋 가운데 무게가 가장 큰 게 승인 단계입니다. 현장을 바꾸는 명령으로 곧바로 이어지는 자리니까요. 그래서 승인부터 보고, 그다음에 질문, 마지막에 멈춘 작업 닫기 순서로 가겠습니다.

[SLIDE 8] 승인과 거절이 남기는 결과

들어가기 전에 말 하나만 정리할게요. 처리 건은 시작부터 종료까지 상태와 기록을 묶어 관리하는 업무 한 건입니다. 지금 보는 쿨러 사례에서는 경보 하나로 열렸어요. 승인과 거절이 이 처리 건에 무엇을 남기는지 화면에서 견줘 보죠.

왼쪽은 쿨러 사건의 실제 기록입니다. 승인 후 현재 조건 검사도 통과한 경우예요. 승인 직후에 팬 속도 명령이 나갔습니다. 다시 관측해 보니 유온이 회복 기준인 55도 아래로 내려와 있었어요. 그래서 경보가 해제됐고, 그다음에 핀 세척 작업지시가 등록됐습니다. 승인 하나에서 명령, 확인, 후속 작업이 차례로 이어진 거예요.

오른쪽은 거절입니다. 거절할 때는 사유를 꼭 적어야 해요. 사유가 비어 있으면 거절 자체가 받아들여지지 않습니다. 거절이 접수되면 설비 명령도, 업무 처리도, 메일도 하나도 만들어지지 않아요. 다만 거절 사유와 상태 변경도 기록이므로, 아무 기록도 쓰지 않는다는 뜻은 아닙니다. 여기서 0건이라는 것은 설비 명령과 후속 업무 실행, 발송을 말해요. 결과 보고에는 아래 따옴표 안의 문장처럼 남습니다. 담당자가 거절했고, 사유는 무엇이고, 설비 명령은 내지 않았다는 내용이죠. 이 처리 건과 사건은 반려로 닫힙니다. 그렇다고 물리적인 고장까지 해결된 것은 아니에요. 거절로 끝났다는 기록과 설비의 실제 상태는 구별해야 합니다.

거절이라고 해서 아무 일도 없었던 게 아니에요. 누가, 왜 실행하지 않기로 했는지가 기록으로 남는 결정입니다.

[SLIDE 9] 응답이 없을 때: 지연 알림과 대기

담당자가 승인도 거절도 안 하면 어떻게 될까요? 그림 위쪽의 시간 축을 따라가 볼게요. 맨 왼쪽에서 승인 요청이 올라왔는데, 아무도 답하지 않습니다. HYD의 이 흐름에는 지연 알림 기한이 10분으로 정해져 있어요. 수업 환경의 20배속에서는 실제 30초에 해당합니다. 업무상 설정값과 화면에서 기다리는 실제 시간을 구분해 주세요.

10분이 지나면 가운데 종 그림처럼 승인 지연 알림이 나갑니다. 문구는 이래요. 승인 대기 중, 승인하거나 거절할 때까지 기다립니다.

여기서 눈여겨볼 건 아래쪽 띠예요. 알림이 나간 뒤에도 띠가 끊기지 않죠? 승인 작업이 열린 채로 남아 있다는 뜻입니다. 시스템이 대신 승인하거나 거절하지 않아요. 몇 시간이 지나도 스스로 끝나지 않고요. 맨 오른쪽처럼 뒤늦게 들어온 승인도 접수할 수 있습니다. 다만 지금의 조건에서 그 조치를 실행할 수 있는지는 다시 확인합니다. 알림 뒤에 거절해도 그대로 받아들여지고요.

쿨러 사건은 약 13초 만에 승인돼서 이 알림이 나가지 않았어요. 기한이 지났다고 자동으로 승인된다면, 사람이 결정하는 자리를 둔 의미가 없겠죠. 알림은 재촉일 뿐이고, 결정은 여전히 사람의 몫입니다.

[SLIDE 10] 에이전트의 질문: 묻고, 답을 받아, 이어서 처리한다

남은 두 자리 가운데 먼저, 에이전트가 사람에게 묻는 실제 장면을 보죠. 화면 첫 칸을 보면, 요청에 조회할 대상 설비가 빠져 있었습니다. 후보 설비가 여럿이라서, 도구로 조회해 봐도 어느 설비를 말하는지 정할 수가 없었죠.

둘째 칸에서 에이전트는 설비를 짐작하지 않았어요. 설비를 잘못 고른 채 결과를 내면, 담당자는 엉뚱한 설비의 근거를 보고 결정하게 되니까요. 대신 조회할 대상 설비를 선택해 달라고 묻고, 멈췄습니다. 질문에는 두 번째 설비와 세 번째 설비라는 선택지도 붙였고요. 이 질문은 일을 끝낸 결과가 아니라, 결과 대신 내는 요청입니다.

셋째 칸에서 담당자가 두 번째 설비를 골랐습니다. 한 질문에는 한 번만 답할 수 있어요. 이미 답한 질문에 또 답하면 받아들여지지 않습니다.

넷째 칸이 중요합니다. 이 기록에서는 같은 대화의 문맥을 이어서 업무 DB를 조회했고, 선택한 두 번째 설비의 결과를 냈습니다. 프로그램의 실행 위치를 그대로 복원하는 방식과는 구별해야 해요. 시스템이 답을 넣어 에이전트를 다시 실행하는 것이므로, 지시만으로 중복 작업이 절대 없다고 보장할 수는 없습니다. 시스템이 에이전트에게 주는 지시도 그렇게 적혀 있어요. 응답을 반영해서 중단된 지점부터 계속하고, 이미 끝낸 작업은 다시 하지 말라고요.

다만 에이전트가 언제나 묻는 건 아닙니다. 묻는 조건이 정해져 있고, 같은 상황에서도 모델에 따라 묻지 않고 끝낸 기록이 있어요.

[SLIDE 11] 질문으로 채우는 것과 바꾸지 못하는 것

그럼 무엇을 물어도 될까요? 모르는 거라고 뭐든 물어도 되는 건 아닙니다. 화면 왼쪽이 물어도 되는 것이에요. 대상 설비, 수량, 사유처럼 요청에서 빠진 업무 정보죠. 단, 조건이 하나 붙어요. 연결된 도구로 조회해도 확인할 수 없는 것이어야 합니다. 이렇게 받은 답은 그 빈칸을 채우는 입력이 되고요.

오른쪽은 질문으로 바꿀 수 없는 것입니다. 우선 도구로 읽을 수 있는 값은 묻지 않아요. 지금 유온이 몇 도인지는 사람에게 물을 게 아니라 도구로 조회할 값이죠.

그 아래 따옴표 두 줄은 에이전트에게 주는 작업 지시의 원문이에요. 질문을 도구 접근이 막혔을 때나, 설비 조치 승인을 피해 가는 데 쓰지 말라고 적혀 있습니다. 그리고 마지막 줄에서, 답변은 실행 권한을 바꾸지 않는다고 적혀 있어요. 이 문장은 에이전트에게 주는 지침이고, 실제 허용 여부는 서버의 권한 검사와 별도 승인 경로가 통제합니다. 허용되지 않은 도구를 쓰게 해 달라고 물어도, 사람이 괜찮다고 답해도, 권한은 그대로라는 뜻입니다.

그래서 담당자가 답에 진행해도 된다고 적었더라도 승인이 아니에요. 빈칸을 채운 답일 뿐이고, 승인은 승인 단계에서 따로 받습니다.

[SLIDE 12] 멈춘 작업 닫기: 끝나지 않는 작업을 사람이 끝낸다

사람이 끼어드는 마지막 자리는 멈춘 작업을 닫는 일입니다. 화면의 네 단계를 왼쪽부터 따라가 볼게요.

첫째, 작업이 멈춥니다. 지난 회차에서 본 한도 기억나시죠? 한 번 실행에 쓸 수 있는 시간에 상한이 있었고, 결과 형식을 고치는 교정은 두 번까지였어요. 이 한도를 넘기거나 실행이 실패하면 정상 결과를 제출하지 못한 채 작업이 멈출 수 있습니다. 그전에 조회나 판단 제출 같은 일부 행동이 있었을 수도 있으므로, 결과 파일이 없다고 아무 행동도 없었다고 보지는 마세요. 그대로 두면 작업도, 처리 건도 끝나지 않아요.

둘째, 담당자가 처리 기록을 읽어요. 어디까지 진행했고 왜 멈췄는지 확인하는 거죠.

셋째, 사유를 적고 닫습니다. 닫은 사람과 사유가 기록에 남고, 그 작업은 취소로 표시돼요. 다만 닫을 수 있는 건 멈춘 작업뿐입니다. 사람이 할 작업이나 아직 진행 중인 작업은 닫히지 않아요.

넷째, 다른 열린 일이 없으면 처리 건이 끝나요. 이때는 사람이 닫았다는 표시와 함께 끝납니다. 닫기는 업무 상태를 정리하는 행위이고, 이미 실행된 일이 있다면 그것을 자동으로 되돌리는 행위는 아닙니다.

이 마지막 표시가 기록을 읽는 사람한테는 중요해요. 조치를 마쳐서 끝난 처리 건인지, 더 갈 수 없어서 사람이 닫은 처리 건인지가 구분돼야 나중에 거슬러 볼 수 있으니까요.

[SLIDE 13]「보류」가 가리키는 세 가지 상황

지금까지 기다리는 장면이 여러 번 나왔죠. 승인을 기다리고, 답을 기다리고, 멈춘 작업도 있었고요. 이런 기다림을 흔히 보류라고 부릅니다. 그런데 이 시스템에는 사람이 누르는 보류 버튼이 따로 없어요. 대신 보류라고 부를 만한 상황이 세 가지 있고, 누가 미뤘느냐에 따라 할 일이 다릅니다. 표의 가운데 열, 누가 미뤘는지를 보세요.

첫째 줄은 응답 없음이에요. 담당자가 아직 결정하지 않은 경우라, 승인하거나 거절하면 됩니다.

둘째 줄은 에이전트의 보류입니다. 지난 회차에서 봤던, 근거가 모자라서 결과를 내지 않고 멈춘 경우예요. 값이 묵었거나 읽지 못했을 때죠. 이때는 빠진 근거와 멈춘 사유를 먼저 확인하고, 지원되는 재처리 경로로 새 값을 읽어 다시 판단하거나 작업을 닫습니다. 화면의 모든 대기 상태에 같은 재평가 버튼이 있다는 뜻은 아니에요.

셋째 줄은 앞에서 본 에이전트의 질문이에요. 필요한 업무 정보가 없어서 묻고 기다리는 거죠. 담당자는 답하면 되고요.

보류 중이라는 말을 들으면, 누가 왜 미뤘는지부터 확인해야 합니다. 그래야 승인할지, 재평가를 요청할지, 답할지가 정해지거든요.

이렇게 해서 처음 물음에 답이 나왔습니다. 추천안이 올라오면 승인 역할의 담당자가 승인하거나 거절하고, 아무도 결정하지 않으면 알림만 나간 채 기다려요. 여기에 일하는 도중의 질문과 멈춘 작업 닫기까지, 사람이 결정하는 세 자리를 모두 찾았습니다.

[SLIDE 14] 승인 권한 · 결정 기록과 AI의 검토

사람이 개입할 자리를 찾았으니, 이번에는 누가 그 자리에 설 수 있는지 보겠습니다. 승인 화면을 볼 수 있다는 것과 조치를 승인할 권한이 있다는 것은 다르죠. 조치마다 승인 역할을 어떻게 정하는지, 결정할 때 무엇을 남기는지 차례로 보겠습니다. 그 뒤에는 시스템의 자동 검사나 다른 AI의 검토를 사람의 승인과 구별해 볼 거예요.

[SLIDE 15] 승인 역할과 등급: 승인할 수 있는 사람

화면 첫 카드에는 조치마다 필요한 승인 역할이 나옵니다. 팬 최대 운전은 운전원, 쿨러 핀 세척은 설비보전팀장입니다. 같은 사건에서 나온 후보라도 실제로 할 일이 다르니 승인 역할도 다를 수 있어요.

둘째 카드는 HYD가 역할에 붙인 등급입니다. 운전원은 1, 생산관리자와 설비보전팀장, 구매팀장과 품질팀장은 2, 공장장은 3입니다. 이 숫자는 HYD의 권한 정책을 표현하는 값이지, 모든 회사에 적용되는 표준 등급은 아닙니다.

셋째 카드에서 시스템이 확인하는 두 가지를 보세요. 먼저 승인하는 사람이 자신이 선택한 역할의 실제 구성원인지 확인합니다. 역할 이름을 직접 입력했다고 그 역할을 얻는 것은 아니에요. 다음으로 그 역할이 조치에 지정된 승인 역할과 같은지, 아니면 그보다 높은 등급인지 검사합니다.

같은 등급이면 무조건 대신 승인할 수 있다는 뜻도 아닙니다. 설비보전팀장에게 지정된 조치를 같은 등급의 생산관리자가 승인하는 것은, 지정된 역할도 아니고 더 높은 등급도 아니므로 이 규칙으로는 허용되지 않습니다. 반면 더 높은 등급의 역할을 가진 구성원은 승인 후보가 될 수 있죠. 실제 승인에는 조치가 현재 실행 가능한지 등 다른 검사도 통과해야 합니다.

마지막 카드의 회사 결재선은 이해를 돕는 비유입니다. 중요한 것은 사람 이름을 추천안에 적는 데서 끝내지 않고, 조직의 역할 정보와 조치의 요구 권한을 시스템이 대조한다는 점이에요. 거절 역시 이 판단을 결정할 수 있는 권한을 확인하고 받습니다.

[SLIDE 16] 결정 기록: 승인하는 순간 남는 것

권한 있는 사람이 결정을 내렸다면 나중에도 그 결정을 확인할 수 있어야 합니다. 표는 쿨러 사건에서 남긴 결정 기록이에요. 이런 이력을 감사 기록, Audit Trail이라고 부릅니다. 누가 언제 무엇을 결정했는지 거슬러 확인하는 자료라는 뜻입니다.

첫 줄은 승인한 사람과 역할입니다. 이 값은 승인 요청을 검증한 시스템이 남깁니다. 에이전트가 결과에 누군가의 이름을 적는 것으로 채울 수 있는 승인 기록이 아니에요.

둘째 줄은 고른 안입니다. 나중에 추천안이 바뀌어도 당시 무엇을 승인했는지 알 수 있도록 사본을 남기고, 추천한 안을 그대로 골랐는지 다른 안을 골랐는지도 구별합니다. 승인 여부만 참으로 저장하면 무엇을 허락했는지는 알 수 없겠죠.

셋째 줄은 접수 시각과 기다린 시간입니다. 이 사례에서는 요청 뒤 약 13초에 승인이 들어왔습니다. 넷째 줄에는 그때 다시 확인한 값이 있어요. 유온은 약 62도였고, 원격 자동 모드에서 운전 중이었습니다. 추천을 계산한 순간의 값과 승인 시점의 값은 다를 수 있으므로 어느 시점의 근거인지 함께 남기는 겁니다.

다만 기록이 있다는 사실과 결정이 옳았다는 것은 별개입니다. 이 기록은 결정의 경위와 근거를 검토하게 해 줍니다. 값이 바뀐 지금도 그 조치를 실행해도 되는지는 다음 검사에서 확인해야 해요.

[SLIDE 17] 승인 뒤의 재확인: 지금도 실행할 수 있는가

추천과 승인 사이에도 설비는 계속 움직입니다. 그래서 사람이 고른 안을 곧바로 명령으로 바꾸기 전에, HYD는 현재 조건을 다시 확인합니다. 화면의 네 칸을 순서대로 보죠.

첫째는 승인 요청을 받는 자리입니다. 담당자가 어떤 역할로 무엇을 골랐는지 받습니다. 둘째는 원천에서 현재 값을 다시 읽는 자리예요. 유온과 운전 모드, 필요한 업무 값을 확인합니다. 추천 때 본 자료를 그대로 다시 보여 주는 것과는 다릅니다.

셋째는 규칙을 다시 적용하는 단계입니다. 선택한 조치가 지금도 허용되는지, 필수 근거를 읽었는지, 승인할 때 보여 준 조치 내용이 바뀌지 않았는지 확인해요. 값이 조금만 달라도 모두 거절하는 것은 아닙니다. 변한 값이 규칙 판정이나 동의한 조치의 내용에 어떤 영향을 주는지가 중요합니다.

넷째는 넘기거나 멈추는 자리입니다. 조건을 통과하면 승인된 조치를 다음 실행 단계로 넘깁니다. 실행 가능 여부나 중요한 근거가 달라졌다면 명령을 내지 않고 다시 판단해 사람에게 검토받도록 합니다. 원천에 연결하지 못해서 확인할 수 없는 경우도 통과로 치지 않아요.

이 검사는 사람이 내린 결정을 대신하지 않습니다. 사람이 승인한 조건이 현재도 성립하는지 확인하는 역할이에요. 승인과 재확인을 통과했더라도 뒤의 명령 전달이나 실제 조치가 실패할 수 있으므로, 실행 결과와 재관측까지 따로 확인해야 합니다.

[SLIDE 18] 가드레일: 추천안이 올라가기 전의 자동 검사

방금 본 검사는 사람이 안을 고른 뒤 실행에 넘기기 전의 검사였습니다. 사람에게 추천안을 올리기 전에도 자동 검사가 하나 더 있어요. 가드레일입니다.

가드레일은 시스템의 입력이나 출력, 행동이 정한 경계를 지키는지 확인하는 장치를 넓게 부르는 말입니다. 구현에 따라 규칙 검사나 모델 기반 검토 등을 쓸 수 있어요. 여기서 보는 HYD의 추천안 가드레일은 모델에게 다시 의견을 묻는 대신 코드에 정한 규칙을 검사합니다.

화면의 둘째 줄이 그 검사 항목입니다. 후보를 고른 규칙의 인용이 있는지, 참고용 절차가 실행할 안으로 섞이지 않았는지 봅니다. 추천한 조치가 제외 규칙에 걸리는지, 승인 역할과 SOP 단계가 갖춰졌는지도 확인합니다. 예를 들어 추천할 수 없는 참고 절차가 끼어 있다면 그 상태로 담당자에게 올리지 않는 거예요.

셋째 줄처럼 위반한 추천안은 제출을 거부합니다. 다만 검사 항목을 통과했다고 인용한 근거의 의미까지 모두 맞거나, 현실의 모든 위험이 없다고 증명된 것은 아닙니다. 자동 검사가 무엇을 확인하는지 알아야 통과의 의미도 정확히 읽을 수 있어요.

[SLIDE 19] 가드레일의 통과, AI의 검토, 사람의 승인

그림에는 추천안이 실행으로 가기 전에 통과하는 두 자리가 있습니다. 하나는 가드레일, 다른 하나는 사람의 승인입니다. 아래에 따로 적힌 AI의 검토도 함께 구별해 볼게요.

가드레일 통과는 정해 둔 자동 검사에서 위반이 발견되지 않았다는 뜻입니다. 다른 에이전트가 초안을 검토해 문제없다고 말한 것은 그 검토가 내놓은 의견이에요. 확인한 근거와 빠뜨린 부분을 따로 읽어야 합니다. AI의 의견만으로 시스템의 제출 조건을 통과했다고 보거나 승인됐다고 처리하지 않습니다.

HYD에서 실행을 허락하는 결정은 권한 있는 담당자의 승인입니다. 담당자는 추천 이유와 대안, 근거의 한계까지 보고 결정해야 해요. 둘 이상의 AI가 같은 의견을 냈어도 사람의 승인 항목이 자동으로 채워지는 것은 아닙니다.

그림은 제출 검사와 승인이라는 역할을 간단히 나눈 것입니다. 실제 실행에는 방금 본 현재 조건 재검사와 실행 단계의 확인도 붙습니다. 따라서 승인 기록은 필요한 결정 근거지만, 그것 하나로 모든 실행 조건이 갖춰졌다고 결론 내리지는 마세요.

이렇게 권한은 누가 결정할지를 정하고, 기록은 무엇을 결정했는지를 남기며, 자동 검사는 정해진 조건을 확인합니다. 이 셋을 나누어야 검토를 승인으로 잘못 읽지 않을 수 있습니다.

[SLIDE 20~21] 실습: 승인과 거절이 처리 결과를 바꾸는 과정 확인

지금까지 사람이 결정하는 세 자리와 승인할 수 있는 역할, 그리고 승인 앞뒤의 두 관문을 따라왔죠. 이제 그걸 직접 해 볼 차례입니다. 첫 실습에서 무엇을 하고, 무엇을 확인하면 되는지부터 정리해 볼게요.

화면 표의 첫 줄부터 볼게요. 처리 건과 내 작업함에서 승인을 기다리는 건을 열고, 올라온 추천안과 그 조치의 승인 역할을 먼저 확인하세요. 그다음 권한이 있는 역할로 승인하거나, 사유를 적어서 거절합니다. 결정한 뒤에는 처리 기록을 열어, 흐름의 상태가 어떻게 바뀌었는지 읽어 보세요.

셋째 줄에서는 지정된 역할도 아니고 더 높은 등급도 아닌, 권한 없는 역할로 결정을 시도합니다. 그 요청이 받아들여지지 않는지 확인하세요.

성공 기준은 표의 오른쪽 열이에요. 대기 건의 승인 역할을 말할 수 있고, 결정에 따라 흐름의 상태가 어떻게 바뀌었는지 처리 기록에서 짚을 수 있으면 됩니다. 권한 없는 결정이 받아들여지지 않은 까닭은 앞에서 본 두 가지 검사로 설명하고요. 그리고 승인한 건과 거절한 건에서 명령과 작업 지시가 생겼는지, 처리 건이 어떤 모양으로 끝났는지 그 차이를 설명할 수 있어야 해요.

이것이 첫 실습입니다. 다음 실습은 정보를 묻는 일과 AI끼리 검토하는 일을 나누어 확인합니다.

[SLIDE 22~23] 실습: 정보가 부족하면 질문하고 다시 처리하는 에이전트

두 번째 실습은 사람에게 묻는 에이전트를 만드는 활동입니다. 첫 실습이 사람의 결정과 상태 변화를 보는 활동이라면, 이번에는 에이전트 쪽에서 정보의 빈칸을 다룹니다. 정보가 빠진 요청을 받으면 짐작하지 않고 확인 질문을 내는 에이전트예요. 지난 회차에 만든 에이전트에 요청 확인 단계를 더해 자연어로 수정을 요청합니다. 여기서 바꾸는 것은 요청 처리 방식이며 온톨로지나 SOP를 다시 만드는 것이 아닙니다.

표의 첫 줄처럼 설비나 경보 패턴, 사례 정보가 빠진 요청을 에이전트에 넣겠습니다. 목표는 에이전트가 빠진 값을 짐작해서 채우지 않는 거예요. 대신 무엇이 왜 필요한지 묻는 질문을 내야 합니다. 답을 넣으면 같은 요청을 다시 판단하게 하고, 질문하기 전과 답변한 뒤의 상태를 기록으로 남기겠습니다. 온톨로지와 SOP는 새로 만들지 않고, 이미 있는 것을 그대로 씁니다.

성공 기준은 표의 오른쪽 열이에요. 다시 판단할 때 답한 값만 반영하고, 답에 승인 같은 말이 섞여도 그걸 승인으로 받아들이지 않으면 됩니다. 질문 전과 답변 뒤의 상태 변화는 기록으로 대조할 수 있어야 하고요.

승인권자에 대한 정보를 답으로 받아도 실제 권한은 역할 정보와 대조해야 합니다. 답에 이름을 적었다는 이유로 권한을 만들어 주면 안 되겠죠.

[이론 04] 실전반 5회차

[SLIDE 24] 요구사항 변경과 바이브 코딩

이번 시간에는 요구사항이 바뀌어도 기존 기능을 지키는 방법을, 변경 요청·명세·계약·검증의 네 단원으로 다룹니다. 앞의 두 실습에서 남긴 승인 기록과 질문 전후 상태를 떠올려 보세요. 이렇게 기능을 더할 때마다 변경할 때 확인할 물음이 따라옵니다. 원래 되던 일은 여전히 될까요? 세 번째 단원, 요구사항 변경과 바이브 코딩은 여기서 출발해요. 쿨러 사건의 에이전트는 지금 잘 동작하고, 그 결과를 사람이 승인하는 자리도 정해져 있습니다. 여기에 새 요구가 딱 한 문장 붙으면, 무엇이 흔들릴 수 있을까요? 요청을 그대로 AI에게 맡기는 방식과, 그 방식이 남기는 빈자리를 따라가 보겠습니다.

[SLIDE 25] 잘 동작하는 에이전트에 붙은 새 요구 한 문장

새 요구가 하나 들어왔다고 해 볼게요. 화면 가운데 문장입니다. 긴급한 경보는 빨리 처리되게 해 달라는 거죠. 지금 에이전트는 경보를 받고, 원인을 가리고, 추천안을 내고, 승인을 기다립니다. 이 흐름의 어디를 바꾸라는 걸까요?

뜻은 알 것 같죠. 그런데 막상 구현하려고 보면 정해지지 않은 게 세 가지 있어요. 아래 세 칸입니다. 칸마다 물음표 아이콘이 붙어 있죠. 이 문장에 적혀 있지 않은 것이라는 표시예요.

첫째, 긴급의 기준이 없습니다. 어떤 경보가 긴급한지 정해지지 않았어요. 같은 말을 들어도 사람마다 떠올리는 경보가 다를 수 있죠.

둘째, 빨라지는 방법이 맡겨져 있어요. 무엇을 바꿔서 빠르게 할지는 받는 쪽이 정해야 합니다.

셋째, 그대로 둘 것이 빠져 있어요. 건드리면 안 되는 것에 대해서는 아무 말이 없습니다.

셋 가운데 가장 무거운 건 셋째예요. 요청한 사람한테는 너무 당연해서 안 적은 거거든요. 승인 없이 설비를 움직이면 안 된다는 것까지 굳이 말하지는 않잖아요. 이번 회차 앞에서 본 것들, 승인 역할을 가리는 검사나 거절하면 명령이 나가지 않는 동작도 모두 이 그대로 둘 것에 들어갑니다. 하지만 받는 쪽은 그 당연한 것까지 구현할 수 있는 형태로 풀어야 합니다. 그럼 이 문장을 그대로 AI에게 주면 어떻게 될까요?

[SLIDE 26] 바이브 코딩: 자연어 요청으로 구현을 맡기는 방식

자연어로 원하는 것을 말하고 AI가 만든 코드를 실행해 보며 고쳐 가는 방식을 바이브 코딩이라고 부릅니다. 이 말은 쓰이는 범위가 넓습니다. 여기서는 상세한 명세와 체계적인 코드 검토 없이, 실행 결과를 보며 요청을 이어 가는 경우를 살펴볼게요. AI에게 자연어로 부탁하는 모든 개발이 이런 방식이라는 뜻은 아닙니다.

첫 줄처럼 빠르게 시도하고 결과를 볼 수 있다는 장점이 있습니다. 다만 작은 시제품이어도 입력 정보나 외부 연결에 따라 영향이 달라지므로, 작다는 이유만으로 검증이 불필요해지는 것은 아니에요.

둘째 줄은 요청에서 빠진 부분입니다. 출력 형식, 예외 처리, 기존 동작의 유지 조건을 적지 않으면 AI가 문맥과 코드로 추정해 채울 수 있어요. 질문으로 확인할 수도 있지만, 반드시 그렇게 한다고 기대할 수는 없습니다.

셋째 줄의 기존 시스템에서는 이 추정이 특히 문제가 됩니다. 결과를 읽는 다른 프로그램과 지켜야 할 승인 규칙이 이미 있기 때문이에요. 한 번 실행해 원하는 표시가 나왔다는 것만으로 그 약속까지 지켰다고 볼 수는 없습니다.

[SLIDE 27] 한 문장의 세 가지 해석

그 한 문장을 구현하려고 보면 해석이 갈립니다. 왼쪽 말풍선이 변경 요청이고요, 여기서 오른쪽으로 세 갈래가 나와요. 갈래 끝마다 그림이 하나씩 있으니, 그림과 같이 보세요.

위쪽은 출력 형식입니다. 끝에 있는 결과 문서를 보면 줄 하나가 달라져 있죠. 결과에 긴급하다는 표시만 더할 수도 있어요. 그러면서 기존 항목의 이름이나 뜻까지 바꿀 수도 있고요.

가운데는 승인 조건이에요. 그림도 사람을 건너뛰어 가는 길로 그려져 있습니다. 빨리 처리하라는 말을, 승인을 기다리지 말라는 뜻으로 읽을 수 있거든요. 승인하는 역할을 낮추라는 뜻으로 읽을 수도 있고요. 그러면 앞에서 본 등급의 경계가 무너집니다. 설비를 멈추고 하는 핀 세척을, 급하다는 이유로 운전원이 승인하게 되는 거죠.

아래쪽은 기존 요청의 처리입니다. 이전 문서와 달라진 문서가 그려져 있어요. 긴급하지 않은 경보는 이전과 같은 결과를 내야 하죠. 그런데 고치는 김에 그 결과까지 달라질 수 있어요.

이 셋은 실제로 일어난 일이 아니라, 일어날 수 있는 해석입니다. 문제는 셋 다 터무니없는 해석이 아니라는 거예요. 요청 문장만으로는 어느 쪽이 맞는지 정할 수가 없습니다. 특히 가운데를 보세요. 앞에서 공들여 본 승인이라는 관문을, 구현을 잘못 바꾸면 훼손할 수 있는 자리예요. 기존 HYD는 승인 없는 실행 경로를 등록 단계에서도 검사하므로, 모호한 요청 하나만으로 실제 승인이 곧바로 우회된다는 뜻은 아닙니다.

[SLIDE 28] 결과 항목을 이름으로 읽는 곳

세 갈래 가운데 하나만 끝까지 따라가 볼게요. 출력 형식이 흔들리면 무슨 일이 생길까요? 에이전트가 낸 결과는 혼자 쓰이지 않습니다. 화면의 파이프라인처럼, 뒤에서 여러 곳이 읽어요.

첫 칸은 결과 객체입니다. 지난 회차에서 본 추천안의 다섯 항목이죠.

둘째 칸은 형식 검사예요. 정해진 이름의 항목이 다 있는지 봅니다. 없으면 에이전트한테 형식을 고쳐서 다시 내라고 돌려보내요.

셋째 칸은 사건 기록입니다. 결과 가운데 진단 가이드 카드를 사건에 싣죠. 이 카드는 나중에 승인할 수 있는 조치의 범위가 됩니다. 카드가 없으면 사건이 설비 명령의 승인을 받지 않아요.

넷째 칸은 승인 화면이에요. 판단 번호가 든 항목을 이름으로 찾아서, 그 번호로 추천안을 불러와 담당자에게 보여 줍니다.

세 곳 모두 항목을 이름으로 읽습니다. 프로그램은 정해진 이름 그대로만 찾아요. 한 글자만 달라도 없는 항목이 되는 거죠. 긴급 표시를 넣다가 이 이름 가운데 하나를 바꿨다고 해 보죠. 고친 사람 눈에는 이름 하나 다듬은 것뿐입니다. 사건 기록과 승인 화면은 에이전트 바깥, 시스템 쪽에 있어서, 에이전트의 코드만 들여다봐서는 누가 이 이름을 읽는지가 보이지 않거든요. 그 항목을 사용하는 단계는 기대한 값을 찾지 못할 수 있어요. 형식 검사에서 먼저 막히면 뒤 단계에 도달하지 않을 수도 있고요. 그렇다면 이런 일은 고친 뒤에 돌려 보면 알 수 있지 않을까요?

[SLIDE 29] 새 기능만 실행해 본 확인이 놓치는 것

물론 고친 뒤에 돌려 보기는 하죠. 그리고 모호한 요청으로 고쳤다고 결과가 꼭 틀리는 것도 아닙니다. 문제는 무엇을 돌려 보느냐예요.

왼쪽이 실행해 본 것입니다. 긴급한 경보 한 건을 넣었더니 추천안이 나오고, 긴급하다는 표시도 보여요. 요청한 기능은 된 것 같죠. 고친 사람은 대개 여기서 끝났다고 봅니다.

오른쪽 칸은 아이콘의 눈에 빗금이 그어져 있죠. 실행해 보지 않은 것이에요. 긴급하지 않은 경보의 결과 항목이 이전과 같은지, 추천안이 여전히 승인을 기다리는지, 하지 않기로 한 일을 여전히 하지 않는지요. 앞에서 본 세 갈래와 그대로 겹치죠.

왼쪽은 요청 문장에 적혀 있었어요. 오른쪽은 적혀 있지 않았고요. 무엇이 그대로여야 하는지 적어 둔 곳이 없으니, 실행해 볼 생각조차 못 하는 겁니다. 새 기능이 된다는 것과 기존 기능이 그대로라는 것, 이 둘은 따로 확인해야 하는 일이에요.

[SLIDE 30] SDD: 명세 주도 개발

그렇다면 무엇이 그대로여야 하는지는 언제, 어디에 적어 둬야 할까요? 네 번째 단원, 명세 주도 개발이 여기에 답합니다. 고친 다음이 아니라, 고치기 전에 문서로 정해 두는 거예요. 그 방식이 무엇인지에서 출발해서, 바이브 코딩과 무엇이 다른지, 그 문서에 무엇을 적는지까지 짚어 보겠습니다.

[SLIDE 31] SDD(Spec-Driven Development)와 명세

SDD, Spec-Driven Development는 요구사항, 입력과 출력의 계약, 금지 조건, 수용 기준을 구현하기 전에 명세로 정하는 개발 방식입니다. 우리말로는 명세 주도 개발이라고 해요. 이름 그대로 명세가 개발을 이끈다는 뜻이죠. 구현할 때도, 다 됐는지 확인할 때도 이 명세가 기준입니다. 정의 안에 낯선 말이 몇 들어 있죠. 계약, 금지 조건, 수용 기준은 뒤에서 하나씩 풀어 볼 거예요.

둘째 줄이 명세예요. 만들 것이 무엇을 받아서, 무엇을 내고, 무엇을 하지 않는지를 적은 문서입니다. 여기서 중요한 건, 사람과 AI가 같은 문서를 본다는 점이에요. 사람은 이 문서로 무엇이 맞는지 판단하고, AI는 이 문서를 받아서 구현합니다.

셋째 줄은 그 문서를 두는 곳이에요. 프로젝트 안에 spec.md 같은 파일로 두고 코드와 함께 변경을 관리할 수 있습니다. 파일 이름이나 다섯 항목의 틀이 모든 SDD 도구에 공통인 의무는 아니에요. 이 수업에서 쓸 명세의 구성입니다. 이어질 명세 실습에서 직접 고칠 파일도 이 이름이에요.

[SLIDE 32] 요청에서 코드까지의 순서: 바이브 코딩과 SDD

바이브 코딩과 SDD, 뭐가 다를까요? AI를 쓰느냐 마느냐가 아닙니다. 순서예요. 두 칸의 꼬리표를 보세요. 왼쪽은 요청에서 바로 코드로 가고, 오른쪽은 요청에서 명세를 거쳐 코드로 갑니다.

왼쪽은 명세를 남기지 않고 요청 한 문장으로 바로 구현하는 경우예요. 해석은 AI가 그때그때 정하고요. 실행해 보고 마음에 안 들면 다시 요청하는데, 요청할 때마다 해석이 또 달라질 수 있습니다. 기존 기능을 지킬 조건을 기록하지 않았다면, 다음 변경 때 다시 확인하기가 어렵습니다.

SDD에서는 요청을 받으면 명세부터 고칩니다. 코드를 고쳐 놓고 문서를 나중에 맞추는 게 아니에요. AI가 명세 초안을 도울 수도 있지만, 무엇을 만들고 지킬지는 사람이 검토해 확정합니다. 구현과 검증은 그 명세를 기준으로 진행하죠. 그래서 달라지는 것과 지킬 것이 문서로 남습니다.

구현은 두 방식 모두 AI에게 맡길 수 있어요. 순서가 바뀌면서 달라지는 건 두 가지입니다. 해석을 누가 정하느냐, 그리고 그 해석이 남느냐예요.

[SLIDE 33] 명세에 적는 다섯 가지

명세에는 무엇을 적을까요? 다섯 가지입니다. 왼쪽 그림이 명세 문서 한 쪽이에요. 다섯 구획으로 칠해 뒀고, 오른쪽에 구획마다 이름을 달아 놨습니다.

맨 위는 요구사항이에요. 무엇을 하는지, 이번에 무엇이 달라지는지 적습니다.

그 아래 둘은 계약입니다. 입력 계약은 어떤 입력을 어떤 형식으로 받는지, 출력 계약은 어떤 항목을 어떤 형식으로 내는지 정해요.

넷째는 금지 조건이에요. 어떤 요구가 더해져도 하지 않을 일입니다.

다섯째는 수용 기준이고요. 무엇이 확인되면 다 된 것으로 볼지를 적습니다.

이름은 딱딱해 보여도 묻는 건 단순해요. 무엇을 하나, 무엇을 받나, 무엇을 내나, 무엇을 안 하나, 언제 끝났다고 보나. 그럼 앞의 그 한 문장, 긴급한 경보를 빨리 처리해 달라는 요청에는 이 가운데 무엇이 들어 있었을까요? 요구사항의 일부뿐이었습니다. 앞에서 가장 무겁다고 한 빈칸, 그대로 둘 것은 입력·출력 계약에도 적고, 아래쪽 금지 조건과 수용 기준에서도 확인할 내용이에요. 오른쪽 이름에서도 금지 조건과 수용 기준을 굵게 강조해 뒀죠.

[SLIDE 34] 계약 · 금지 조건 · 수용 기준

다섯 번째 단원은 계약과 금지 조건, 수용 기준입니다. 한 문장의 요청에 빠져 있던 나머지 넷은 어디서 찾을 수 있을까요? 사실 쿨러 사건의 시스템에서는 이미 여러 장치가 그 일을 하고 있어요. 명세 파일 하나로 모여 있지 않을 뿐이죠. 받는 입력과 내는 출력, 하지 않을 일, 다 되었는지를 가르는 조건 순서로 하나씩 찾아가 보겠습니다.

[SLIDE 35] 계약: 주는 쪽과 받는 쪽이 지키는 항목과 형식

먼저 계약이라는 말부터 풀어 볼게요. 계약, Contract는 데이터를 주는 쪽과 받는 쪽이 지키기로 한 항목과 형식, 뜻과 처리 조건입니다.

그림을 보면 에이전트가 가운데 있습니다. 받기도 하고 내보내기도 하니까, 계약이 둘이에요.

왼쪽이 입력 계약입니다. 처리 건이 에이전트에게 무엇을 주는지 정하죠. 위에 적힌 것처럼 설비, 경보, 사건 번호가 들어와요.

오른쪽이 출력 계약이에요. 에이전트가 무엇을 내는지 정하고, 승인 화면과 뒤 단계가 그걸 읽습니다. 원인, 추천, 판단 번호가 나가죠.

아래 받침선은 두 계약을 묶어 계약이라고 이름 붙인 자리예요. 화면 위 메시지처럼, 기존 계약을 깨는 변경을 한쪽에만 적용하면 다른 쪽이 읽거나 처리하지 못할 수 있습니다. 앞에서 본 이름 바꾸기가 바로 이 얘기예요. 그래서 계약은 에이전트 혼자 지키는 게 아닙니다. 양쪽이 함께 지키는 거예요.

거꾸로 보면 계약은 자유도 줍니다. 외부에 약속한 항목의 이름과 형식, 뜻과 동작을 유지하면 안쪽의 도구나 구현 방식을 바꿀 여지가 생깁니다. 다만 호출 순서를 바꿔 중복 실행이 생기거나 권한과 응답 조건이 달라지면, 겉모양이 같아도 같은 계약을 지켰다고 할 수 없어요. 무엇을 고쳐도 되고 무엇은 함부로 고치면 안 되는지, 그 경계를 계약이 그어 줍니다.

[SLIDE 36] 입력 계약의 실제: 쿨러 성능 저하 경보의 입력

입력 계약이 실제로 어떻게 생겼는지 볼게요. 쿨러 성능 저하 시나리오의 다른 실행에서 에이전트가 받은 입력입니다. 앞에서 본 승인 시각의 기록과 같은 한 건으로 합쳐 읽지 마세요.

왼쪽 코드를 보면 항목이 셋이에요. 첫째는 대상 설비입니다. 유압 설비 HYD-01이죠.

둘째는 경보예요. 안에 경보 번호와 경보의 유형이 들어 있습니다. 유형은 쿨러 성능 저하고요. 번호의 뒷자리와 그 아래 다른 항목은 화면에서 줄여 뒀어요.

셋째는 이 경보가 속한 사건의 번호입니다.

오른쪽 넷째 짚을 점을 보세요. 유온이나 냉각 효율 같은 값은 입력에 없습니다. 왜일까요? 이런 값은 계속 변하니까, 에이전트가 판단하는 순간에 도구로 직접 읽거든요. 경보 때의 유온을 입력에 담는 설계도 가능하지만, 그 값을 쓰려면 관측 시각과 최신성 조건을 함께 정해야 합니다. 오래된 값을 현재 값처럼 쓰면 문제가 되겠죠. 앞에서 승인할 때 값을 다시 읽었던 것과 같은 까닭이죠. 이 HYD 입력 예는 주로 무엇에 대해 일할지 정하고, 변하는 값은 별도로 조회하는 선택을 한 것입니다. 모든 입력 계약이 값 자체를 제외해야 하는 것은 아니에요.

[SLIDE 37] 출력 계약: 항목마다 정하는 이름, 형식, 뜻

이번엔 출력 계약입니다. 다섯 항목마다 세 가지를 정해요. 이름, 형식, 그리고 뜻입니다. 표의 세 열이 바로 그거예요.

첫째와 둘째 줄은 원인과 고장 유형입니다. 형식은 지식 그래프의 식별자예요. 이 기록에서는 쿨러 핀 오염과 쿨러 냉각 성능 상실을 가리키는 식별자가 들어갔습니다.

셋째 줄은 진단 결과의 가이드 카드예요. 앞에서 본 것처럼, 사건 기록에 실려서 승인할 수 있는 조치의 범위가 됩니다.

넷째 줄이 판단이에요. 강조해 둔 줄이죠. 안에 1순위로 추천하는 조치, 그 이유를 적은 설명, 후보의 순서가 들어 있습니다. 이 기록의 추천은 팬 최대 운전이었어요.

다섯째 줄은 판단을 제출하고 돌려받은 번호입니다. 승인 화면이 이 번호로 추천안을 찾죠.

이렇게 적어 두면 변경을 두 가지로 가를 수 있어요. 긴급 여부를 새 항목으로 더하는 건 계약을 넓히는 변경입니다. 기존 소비자가 추가 항목을 허용하고 기존 항목의 의미와 동작이 유지된다면 호환될 수 있어요. 알 수 없는 항목을 거부하는 검사나 고정된 결과 비교가 있다면 추가만 해도 실패할 수 있으므로 실제 소비자와 함께 확인해야 합니다. 반대로 추천 항목의 이름이나 뜻을 바꾸는 건 계약을 깨는 변경이에요. 이름이 바뀌면 읽던 쪽이 더는 못 읽습니다. 이렇게 읽던 쪽을 깨뜨리는 변경을 업계에서는 Breaking Change라고 부릅니다. 그 둘을 가르는 기준이, 이 표에 적힌 이름과 형식과 뜻이에요.

그럼 이름은 그대로 두고 뜻만 바꾸면 어떻게 될까요? 추천이라는 이름은 둔 채, 그 안에 1순위 조치 대신 가장 빨리 끝나는 조치를 넣기로 했다고 해 보죠. 빨리 처리하라는 말을 그렇게 읽을 수도 있으니까요. 승인 화면은 아무 문제없이 읽어서, 그 조치를 1순위처럼 담당자에게 보여 줍니다. 이름이 바뀌면 어디선가 멈추기라도 하는데, 뜻이 바뀌어도 형식 검사에서는 드러나지 않을 수 있어요. 의미와 기대 행동을 확인하는 검사가 필요한 이유입니다. 표에 셋째 열, 뜻을 따로 적어 두는 까닭입니다. 그래야 이런 변경도 계약을 깬 것으로 가려낼 수 있어요.

[SLIDE 38] 금지 조건의 실제: 작업 규칙의「절대 금지」

계약 다음은 하지 않을 일입니다. 쿨러 사건의 에이전트가 읽는 작업 규칙에는 절대 금지라는 절이 따로 있어요. 왼쪽이 그 원문입니다. 세 줄을 차례로 볼게요.

첫째 줄은 설비 명령, 업무 시스템의 조치 실행, 그리고 사람 대신 승인하는 일을 금지합니다. 사람이 승인하기 전에는 어떤 조치도 실행하지 않는다고 덧붙어 있죠. 첫 단원에서 본 그 경계를, 에이전트가 읽는 글에도 적어 둔 거예요.

둘째 줄은 온톨로지의 스키마와 규칙, 스킬을 고치지 못하게 해요. 지식 그래프는 조회 도구로 읽기만 합니다. 이 줄의 까닭은 가드레일을 떠올리면 보여요. 가드레일이 대어 보던 제외 규칙도, 조치마다 붙은 승인 역할도 모두 이 지식 그래프에 들어 있었죠. 에이전트가 그걸 고칠 수 있다면, 자기를 검사할 기준을 자기가 바꾸는 셈이 됩니다.

셋째 줄은 판단 제출 도구에 관한 거예요. 지난 회차 도구 목록에서 본 그 도구죠. 이 도구는 고를 안을 올리는 일만 허용하고, 설비나 업무 조치의 승인이나 실행이 아니라고 못 박습니다. 이름에 제출이 들어 있으니까, 실행으로 오해하지 않게 적어 둔 거예요. 이런 글은 업무 지침이고, 실제 금지는 허용 도구 목록과 서버 권한, 승인 경로의 검사로 뒷받침해야 합니다.

눈여겨볼 건, 이 절이 원인을 가리고 조치를 견주는 업무 절차와 따로 적혀 있다는 점입니다. 절차는 해야 할 일이고, 이 절은 하지 않을 일이에요. 왜 굳이 따로 적었을까요?

[SLIDE 39] 요구사항은 늘어나고 금지 조건은 남는다

따로 적는 까닭은, 요구사항과 금지 조건이 시간이 지나면서 바뀌는 방식이 다르기 때문이에요.

왼쪽 요구사항에는 해야 할 일을 적습니다. 변경 요청이 올 때마다 줄이 늘어나요. 원인을 가리고 조치 후보를 견주어 추천안을 낸다는 원래 줄이 있고, 여기에 긴급한 경보에는 긴급 여부를 함께 낸다는 줄이 새로 붙죠.

오른쪽 금지 조건에는 어떤 요구가 더해져도 하지 않을 일을 적어요. 이번 기능 추가에서는 그대로 유지해야 할 조건입니다. 정책 자체를 바꿀 필요가 있다면 별도의 책임 있는 검토와 승인을 거쳐 바꾸는 것이지, 기능 추가에 묻어서 없애지 않습니다. 예로 적힌 줄을 보세요. 사람이 승인하기 전에는 어떤 조치도 실행하지 않는다. 방금 본 그 문장이죠.

둘을 섞어 적으면, 어느 줄은 늘려도 되고 어느 줄은 지켜야 하는지 가리기 어려워요. 따로 적어 두면 이렇게 작동합니다. 긴급한 경보를 빨리 처리하라는 요구가 와도, 명세에는 승인 전에는 어떤 조치도 실행하지 않는다고 적혀 있죠. 그러면 승인 생략이라는 해석이 명세에 어긋난다고 가릴 수 있습니다. 실제 구현이 그 금지를 지켰는지는 실행되는 검사로 확인해야 해요. 다음에 다른 요구가 붙어도 마찬가지예요. 요구사항의 줄은 그 요구 하나에만 쓰이지만, 금지 조건 한 줄은 앞으로 붙을 모든 요구에 똑같이 걸립니다.

[SLIDE 40] 수용 기준: 다 되었는지를 가르는 조건

마지막은 수용 기준입니다. 수용 기준, Acceptance Criteria는 구현이 요구를 채웠다고 받아들이는 조건이에요. 쉽게 말하면, 무엇을 보고 다 됐다고 말할 거냐는 문제입니다.

첫째 줄의 둘째 문장이 핵심이에요. 실행해 보고 참인지 거짓인지 가를 수 있는 문장으로 적어야 합니다. 기준이 없으면 어떻게 될까요? 만든 사람이 된 것 같다고 하면 끝이죠. 기준이 있으면 다릅니다. 적어 둔 문장이 하나하나 참으로 나와야 끝이에요.

둘째 줄은 무엇을 적느냐입니다. 어떤 상황에서 어떤 결과가 나와야 하는지를, 한 문장에 하나씩 적어요.

셋째 줄은 언제 쓰이느냐예요. 구현하기 전에는 무엇을 만들지의 기준이 되고, 구현한 뒤에는 다 됐는지 확인하는 항목이 됩니다. 같은 문장이 두 번 일하는 거죠.

그래서 수용 기준은 구현한 뒤가 아니라 구현하기 전에 적습니다. 구현한 뒤에 적으면 이미 나온 결과에 기준을 맞추게 되거든요. 그렇게 되면 잘못 나온 결과도 정답으로 인정할 위험이 생깁니다. 이미 구현한 뒤라도 원래 요구와 근거를 바탕으로 기준을 세울 수 있지만, 현재 결과를 그대로 정답으로 옮겨 적어서는 안 됩니다.

[SLIDE 41] 판정할 수 있는 문장과 판정할 수 없는 문장

참과 거짓으로 가를 수 있게 적는다는 게, 생각보다 까다로워요. 양쪽 문장을 나란히 놓고 보죠. 왼쪽부터요. 거절이 잘 처리된다. 긴급한 경보가 빨리 처리된다. 그럴듯하죠? 그런데 잘은 어느 정도이고, 빨리는 얼마일까요?

이 문장에는 판정할 기준이 없습니다. 그래서 실행한 사람마다 판정이 달라져요. 어떤 사람은 잘 됐다고 하고, 어떤 사람은 아니라고 하겠죠.

오른쪽은 다릅니다. 거절하면 설비 명령이 0건이다. 실행한 뒤에 명령 수를 세면 돼요. 0이면 참, 하나라도 있으면 거짓이죠. 긴급한 경보의 추천안도 승인 대기 상태가 된다. 이건 처리 건의 상태를 대조하면 됩니다. 같은 입력과 조건에서 같은 항목을 확인하면 판정을 일관되게 할 수 있어요.

오른쪽 문장에는 숫자나 상태의 이름처럼, 세거나 견줄 대상이 들어 있죠. 수용 기준을 쓰고 나면 스스로 물어보세요. 무엇을 세면 되지? 무엇을 무엇과 견주면 되지? 답이 안 나오면, 상황과 기대 결과를 적은 문장으로 다시 쓰면 됩니다.

[SLIDE 42] 승인 단계가 지키는 수용 기준

그런데 거절하면 설비 명령이 0건이라는 문장, 어디서 본 것 같지 않나요? 첫 단원에서 승인 단계를 다룰 때 이미 봤습니다. 그 승인 단계의 동작은 우연히 그렇게 된 게 아니에요. 상황마다 확인되어야 하는 결과가 정해져 있고, 저장소에는 준비한 입력과 실행 대역으로 이 결과를 확인하는 자동 시험이 있습니다. 오늘 학생 계정의 실제 실행을 새로 확인했다는 뜻은 아닙니다. 표의 다섯 줄이 바로 그거예요.

첫 줄, 이 시험의 실행 조건에서 권한 있는 담당자가 승인하면 승인한 사람과 역할이 기록되고, 처리 건이 끝까지 처리됩니다.

둘째 줄은 강조해 둔 줄이죠. 사유를 적고 거절하면 설비 명령과 업무 처리가 0건이고, 처리 건은 반려로 닫혀요.

셋째 줄, 사유 없이 거절하거나 권한 없는 역할로 거절하면, 그 요청 자체가 받아들여지지 않습니다.

넷째 줄, 응답이 없으면 지연 알림만 나가고 처리 건은 계속 기다려요.

다섯째 줄, 알림 뒤에 승인해도 현재 실행 조건을 충족하면 정상으로 처리됩니다.

다섯 줄 모두 이런 상황이면 이런 결과라는 꼴이죠. 앞에서 말로 들은 승인 규칙이, 실제로는 이렇게 판정할 수 있는 문장으로 적혀 있는 겁니다. 그래서 승인 단계를 고치다가 이 가운데 하나라도 달라지면, 그 조건을 실제로 검사하는 시험을 실행하면 차이를 찾을 수 있습니다. 시험에 없는 모든 경우까지 보장하는 것은 아니에요.

오른쪽 구분 열도 한번 보세요. 일이 순조롭게 풀리는 승인은 첫 줄 하나뿐이에요. 나머지 넷은 거절하거나, 사유나 권한을 빠뜨리거나, 답이 없거나, 늦은 경우죠. 잘 되는 경우만 적어 두면, 거절했는데 명령이 나가는 것 같은 사고는 확인할 길이 없습니다. 그래서 수용 기준에는 거절과 누락, 지연처럼 중요한 예외도 적어 둡니다. 빠뜨린 상황이 발견되면 그 기준과 검사를 보완하고요. 이렇게 해서 입력과 출력의 계약, 금지 조건, 수용 기준을 모두 실제 대상에서 찾았습니다.

[SLIDE 43] 명세에 따른 구현과 변경의 확인

명세 부분의 마지막 단원은 명세에 따른 구현과 변경의 확인입니다. 처음의 그 한 문장 기억하시죠? 긴급한 경보는 빨리 처리되게 해 달라는 요청이요. 이걸 명세로 다시 쓰면 어떻게 될까요? 그 명세를 기준으로 구현을 맡기고, 고친 뒤에는 무엇을 다시 실행해야 하는지까지 따라가 보겠습니다.

[SLIDE 44] 명세로 다시 쓴 변경 요청

그 한 문장을 명세로 옮기면 왼쪽처럼 됩니다. 제목처럼 긴급 표시를 추가하는 명세이고, 설명을 위해 써 본 예예요. 다섯 구획을 위에서부터 읽어 볼게요. 여기서는 먼저 긴급 여부를 구분해 표시하는 작은 변경으로 범위를 합의한 예입니다. 처리 속도를 실제로 높인다는 요구까지 완성한 것은 아닙니다.

요구사항에서는 긴급 경보를, 판단할 때 읽은 유온이 60도 이상인 쿨러 성능 저하 경보로 정했습니다. 60도는 운전 화면이 유온 높음으로 경고하는 선이에요. 설비를 멈추는 인터록 선보다는 낮아서, 멈추기 전에 먼저 움직이자는 기준으로 고른 예입니다. 긴급의 기준을 말이 아니라 수치로 정한 거죠. 그리고 긴급 경보의 추천안에는 긴급 여부를 함께 낸다고 적었습니다.

그런데 왜 판단할 때 읽은 값이라고까지 적었을까요? 쿨러 사건에 대어 보면 보여요. 경보가 났을 때 유온은 60도 아래였습니다. 그런데 승인할 때 다시 읽은 값은 60도를 넘었죠. 이 예시 기준에 적용하면 같은 사건도 어느 시점의 값을 쓰느냐에 따라 분류가 달라질 수 있습니다. 실제 기록에 이미 긴급 항목이 구현돼 있었다는 뜻은 아니에요. 잴 때를 적어 두지 않으면, 나중에 수용 기준으로 판정할 수가 없습니다.

입력 계약에는 설비, 경보, 사건 번호의 세 항목을 바꾸지 않는다고 적었어요. 긴급 여부를 가리려고 유온을 입력에 실을 필요는 없습니다. 앞에서 본 것처럼, 유온은 판단하는 순간 도구로 읽으니까요.

출력 계약에서는 먼저, 기존 다섯 항목의 이름과 형식을 바꾸지 않는다고 적었어요. 그다음에 긴급 여부를 담는 항목 하나를 참이나 거짓으로 더합니다. 일반 경보에도 거짓을 내고, 긴급 조건을 만족하면 참을 냅니다. 이 확장을 받는 쪽이 허용하는지 먼저 확인하고 출력 검사도 맞춰야 합니다.

금지 조건에는 두 줄이 있어요. 긴급 경보도 승인 전에는 조치를 실행하지 않는다. 그리고 필수 값을 읽지 못해도 짐작해 채우지 않는다. 둘째 줄은 왜 넣었을까요? 빨리 처리하라는 말은, 값을 기다리지 말고 있는 값으로라도 채워서 빨리 내라는 뜻으로도 읽힐 수 있거든요.

수용 기준에는 세 줄을 적었습니다. 고정된 입력과 조회 자료에서 일반 경보의 기존 항목은 같은 뜻과 처리 결과를 유지한다. 긴급 경보의 추천안도 승인 대기 상태가 된다. 필수 값을 읽지 못한 경보는 결과 없이 보류된다. 마지막 줄은 앞에서 본 에이전트의 보류를, 긴급하다는 이유로 건너뛰지 못하게 지키는 줄이에요.

앞에서 갈렸던 세 갈래를 떠올려 보세요. 출력 형식, 승인 조건, 기존 요청의 처리였죠. 오른쪽 짚을 점처럼, 그 세 갈래가 각각 출력 계약, 금지 조건, 수용 기준의 줄에서 정해졌습니다. 해석을 AI

가 아니라 명세가 정한 거예요. 실제 처리 시간을 줄이는 요구가 남아 있다면, 어느 구간을 얼마나 줄일지와 승인 대기를 어떻게 다룰지 별도로 합의해야 합니다. 표시를 추가한 것만으로 빨라졌다고 결론 내리면 안 되겠죠.

[SLIDE 45] 명세를 기준 문서로 삼는 구현

명세를 쓴 다음, 구현까지는 어떤 순서로 갈까요? 화면의 네 칸입니다.

첫째, 명세를 고칩니다. 무엇이 달라지고 무엇을 지킬지 적어요.

둘째, 구현을 맡깁니다. 이때 AI에게 주는 건 처음의 한 문장이 아니에요. 방금 본 명세입니다. AI는 그 명세를 기준으로 코드를 고치죠.

셋째는 강조해 둔 칸인데요, 수용 기준으로 확인합니다. 기준마다 실행해 보고 참인지 거짓인지 가려요.

넷째, 어긋난 곳을 고칩니다. 이 칸의 아이콘이 되돌아가는 화살표죠. 수용 기준 하나가 거짓으로 나왔다고 해 볼게요. 그러면 둘 중 하나예요. 구현이 명세와 다르면 구현을 고치고, 명세에 빠진 게 있었다면 요구를 다시 확인해 명세를 고칩니다. 검사를 통과시키려고 기대 결과를 현재 구현에 맞춰 낮추는 것은 아닙니다.

한 문장만 줬을 때를 떠올려 보세요. 결과가 마음에 안 들어도, 구현이 틀린 건지 요청이 모자랐던 건지 가릴 기준이 없었죠. 명세가 있으면 그 기준이 문서로 남고, 명세 자체에 빠진 것도 드러나요. 그래서 명세는 한 번 쓰고 끝내는 문서가 아닙니다. 고쳐 가는 문서예요.

[SLIDE 46] 명세를 실행되는 검사로 옮긴 두 가지 예

하지만 명세는 문서라서, 안 읽으면 그만이에요. 그래서 명세에 적은 건 실행되는 검사로 옮겨 둬야 힘을 가집니다. 바꾸지 말라고 적어 두는 것과, 바뀌면 실패하는 검사가 있는 건 다르니까요. 화면의 두 가지는 이 시스템에 실제로 있는 장치예요.

왼쪽은 기대 결과의 저장이에요. 같은 경보에 대해 추천안이 어떻게 나와야 하는지를 저장해 둡니다. 무언가를 고친 뒤에는 새 결과를 이 저장본과 견줘요. 달라진 곳이 있으면 바로 드러나죠. 저장해 둔 기대 결과는, 결과를 일부러 바꿀 때만 근거와 함께 갱신합니다. 다르게 나올 때마다 새 결과로 덮어쓰면, 저장본이 더는 기준 노릇을 못 하니까요.

오른쪽은 금지 조건의 고정이에요. 쿨러 사건 에이전트의 작업 규칙을 고정해 두고, 기준 파일의 바이트와 달라지면 검사가 실패하게 해 뒀습니다. 이것은 글이 보존됐는지 검사하는 것이고, 에이전트가 실행 중 그 글을 항상 지켰다는 검사는 아닙니다. 앞에서 본 절대 금지 절이, 무언가를 고치는 도중에 빠지는 일을 막는 거예요.

한 글자까지 보는 데에도 까닭이 있어요. 금지 문장은 낱말 하나만 빠져도 뜻이 달라집니다. 사람 대신 승인이라는 말 하나가 지워지면, 그 금지가 통째로 사라지죠. 어떤 고침이 해가 없는지를 검

사가 대신 가려 줄 수는 없어요. 그래서 달라지기만 하면 일단 멈추고, 사람이 보고 판단하게 하는 겁니다.

이런 검사가 있으면 무엇을 확인해야 하는지 사람이 매번 기억하지 않아도 됩니다. 변경 뒤 검사를 실제로 실행하면 저장된 기준과 대조할 수 있으니까요. 자동 검사가 있다는 것과 매 변경마다 실행됐다는 것도 구별해야 합니다.

[SLIDE 47] 변경 뒤의 확인: 기존 요청과 새 요청을 함께 실행한다

구현을 마쳤으면 다시 실행해 봐야죠. 새 요청만이 아니라 기존 요청도 함께요. 화면의 세 카드가 그 세 경우예요.

첫 카드는 기존 요청입니다. 긴급하지 않은 경보를 넣어요. 같은 입력과 고정한 조회 자료에서 기존 항목의 의미와 처리 결과가 유지되고, 승인 대기가 되어야 합니다. 새 긴급 여부 항목은 거짓이어야 해요. 그대로여야 하는 경우죠.

가운데 카드는 새 요청이에요. 판단할 때 유온이 60도 이상인 경보를 넣습니다. 긴급 여부가 참이고, 그래도 승인 대기가 되어야 해요. 이건 달라져야 하는 경우입니다. 긴급하다고 승인을 건너뛰지는 않는 거죠.

셋째 카드는 필수 값이 없는 경우예요. 필수 값을 읽지 못한 경보를 넣습니다. 그러면 결과를 채우지 않고 보류해야 해요. 값을 읽지 못하면 지어내지 않는다는 규칙은, 긴급하든 아니든 같으니까요.

세 카드가 어디서 왔는지도 보세요. 모두 앞에서 쓴 명세에서 나왔어요. 첫 카드는 수용 기준의 첫 줄, 가운데 카드는 요구사항의 긴급 기준과 수용 기준의 둘째 줄, 셋째 카드는 금지 조건의 둘째 줄과 수용 기준의 셋째 줄이죠. 명세에 적은 줄이 그대로 다시 실행해 볼 경우가 된 겁니다.

카드 아래 꼬리표를 보면, 셋 가운데 달라져야 하는 건 하나뿐이에요. 나머지 둘은 그대로여야 하고요. 새 기능만 실행해서는 바로 이 그대로여야 하는 쪽을 놓쳤던 겁니다. 그래서 확인할 때 쓴 입력과 기대 결과는 저장해 둬요. 조회 자료와 비교할 항목도 함께 저장합니다. 판단 번호와 시각처럼 실행마다 달라지는 값은 글자 그대로 같아야 하는 값과 구분해야 해요. 긴급 기준인 60도 바로 아래와 정확히 60도인 경우도 넣으면 경계 조건을 확인할 수 있습니다. 이어서 이 입력과 기대 결과를 검증 시나리오로 다룹니다.

[SLIDE 48~49] 실습: 명세에 따라 기존 기능을 유지하며 요구사항 추가

앞에서 만든 에이전트에 긴급 조건을 추가하겠습니다. 먼저 제공된 모호한 변경 요청의 기존 비교 기록을 읽고, 출력 항목과 승인 조건 중 무엇이 달라졌는지 확인합니다. 이 기록은 이번 실행의 결과가 아닙니다. 모호한 요청이 반드시 실패한다는 뜻도 아니며, 차이가 없으면 없는 그대로 기록합니다.

같은 프로젝트의 변경 전 판본과 입력을 보존한 뒤 spec.md를 먼저 작성하겠습니다. 기존 일반 요청의 처리, 긴급 기준, 필수 값이 없을 때의 보류, 승인 없이 확정하지 않는 조건, 결과 형식과 수용

기준을 적습니다. 그 명세에 맞춰 필요한 기능만 더하고 일반 요청과 긴급 요청을 같은 조건으로 비교합니다.

수용 기준마다 기대와 실제를 나누어 확인하고 입력과 기대 결과, 조회 자료와 판본을 함께 저장하겠습니다. 실행마다 달라지는 번호와 시각은 별도 규칙으로 비교합니다. 이어질 검증 실습에서는 이 자료를 그대로 시나리오와 회귀 확인에 사용합니다.

[이론 06] 실전반 5회차

[SLIDE 50] 기대 행동의 시나리오: BDD와 Given · When · Then

첫 단원은 명세에 적어 둔 약속에서 출발합니다. 승인 전에는 설비 명령을 내지 않는다, 결과는 정해진 항목으로 낸다, 명세에는 이런 약속들이 적혀 있죠. 그런데 이 약속을 어떻게 적어 둬야, 에이전트가 실제로 지켰는지를 실행 결과로 확인할 수 있을까요? 쿨러 사건의 승인 단계에서, 담당자가 추천안을 거절하는 장면 하나를 붙잡고 그 방법을 찾아보겠습니다.

[SLIDE 51] 명세에 적은 것과 실제로 일어나는 것

화면의 그림부터 볼게요. 왼쪽 문서가 명세입니다. 요구사항하고 같이 입력 계약, 출력 계약, 금지 조건, 수용 기준이 한 칸씩 적혀 있죠. 무엇을 받고, 무엇을 내고, 무엇을 하면 안 되고, 언제 다 됐다고 볼지, 이걸 정해 둔 문서예요.

가운데 에이전트가 실제로 실행되면 오른쪽에 결과가 나옵니다. 쿨러 사건이라면 추천안이죠. 그런데 여기서 걸리는 게 하나 있어요. 명세는 문서고, 추천안은 실행해 봐야 나온다는 겁니다. 그러니까 명세를 아무리 잘 적어도, 에이전트가 그대로 움직였는지는 결과를 보기 전엔 알 수 없어요.

명세가 「무엇을 지킬 것인가」에 답했다면, 남은 물음은 「정말 지켰는가」입니다. 그 답이 나오는 곳이 아래쪽 점선이에요. 돋보기가 가리키는 자리, 명세와 실제 결과를 대어 보는 자리죠. 이번 회차에 배울 시나리오와 시험은 모두 이 자리에서 쓰입니다.

[SLIDE 52] 손으로 한 번 확인하는 방식의 한계

화면에서 직접 눌러 확인하면 사용자가 겪는 흐름을 볼 수 있습니다. 다만 한 번 잘 됐다는 기억만으로 변경을 판단하기는 어렵습니다. 승인 단계에도 승인과 거절, 무응답, 권한 없는 요청처럼 여러 경우가 있으니까요.

첫 카드처럼 한 번의 실행은 그중 한 경우를 보여 줍니다. 둘째 카드처럼 기능을 바꾸면 기존 경우도 다시 확인해야 하죠. 그때 무엇을 넣었고 어떤 결과가 맞는지 남기지 않았다면, 셋째 카드처럼 확인하는 사람마다 판단이 달라질 수 있습니다.

손으로 시험해도 입력과 기대 결과를 기록할 수 있습니다. 새로운 문제를 탐색하거나 화면을 이해하기 쉬운지 보는 데도 사람의 확인이 필요해요. 여기서는 반복할 조건을 명확히 남기고 자동으로 다시 확인할 수 있게 만드는 데 초점을 두겠습니다. 사람의 기억에만 기대던 확인을 다음 변경에도 쓸 수 있는 기준으로 바꾸는 겁니다.

[SLIDE 53] BDD: 행동을 시나리오로 적고 실행으로 확인하는 방식

이 문제를 다루는 방식이 BDD, 행동 주도 개발입니다. 업무를 아는 사람과 개발하고 검증하는 사람이 구체적인 사례를 놓고, 어떤 행동이 필요한지 함께 정하는 방식이에요. 먼저 기대를 맞추고, 사례를 시나리오로 적고, 실행 가능한 검증에 연결합니다.

예를 들어 거절한다는 말만 같아도, 한 사람은 승인 화면만 닫힌다고 생각하고 다른 사람은 처리 건까지 종료된다고 생각할 수 있죠. 거절하면 명령은 몇 건이고 처리 상태는 무엇인지 구체적으로 물으면 이 차이가 드러납니다. 코드를 만든 뒤에야 서로 다른 기대를 발견하는 일을 줄이는 거예요.

화면의 행동은 시스템 밖에서 확인할 수 있는 결과를 뜻합니다. 내부에서 어떤 함수를 썼느냐보다, 담당자가 거절했을 때 설비 명령이 생기지 않고 사유가 남는지를 보는 겁니다. 시나리오는 이런 상황과 기대를 구체적인 예로 적은 글이에요.

이번 시간에는 이미 정리한 명세를 바탕으로 시나리오를 만들고 검증에 연결하는 부분을 살펴보겠습니다. 실제 개발에서는 시나리오를 논의하다가 빠진 요구를 찾아 명세를 보완할 수도 있어요.

[SLIDE 54] Given · When · Then: 시나리오의 세 부분

시나리오를 세 부분으로 나누면 무엇을 준비하고 무엇을 확인할지가 보입니다. Given은 행동 전의 전제입니다. 화면에서는 추천안이 올라와 승인 단계가 열려 있는 상태죠. When은 확인할 행동이고, 여기서는 권한 있는 담당자가 사유를 적어 거절하는 일입니다. Then은 그 뒤에 기대하는 결과입니다.

거절 뒤에는 설비 명령과 후속 업무 처리가 없어야 하고, 처리 건은 거절 상태로 닫혀야 합니다. 하나의 거절 행동에도 확인할 결과는 여럿일 수 있어요.

시나리오마다 한 가지 업무 행동에 초점을 맞추면 실패했을 때 원인을 찾기 쉽습니다. 여러 단계가 들어가는 시나리오를 쓸 수 없다는 뜻은 아니에요. 준비 단계와 확인하려는 행동을 섞지 않고, 이 사례에서 무엇을 판단하려는지 드러내는 것이 중요합니다.

[SLIDE 55] 시나리오 한 편: 승인 단계의 거절

앞의 거절을 구조가 보이는 글로 적은 예입니다. 맨 위는 기능 이름, 그 아래는 시나리오 이름이에요. 이어서 전제와 행동, 기대 결과를 적습니다. 이 구조로 시나리오를 쓰는 언어를 Gherkin이라고 합니다.

Then 아래의 And는 앞부분에 같은 종류의 조건을 이어 붙이는 말입니다. 여기서는 설비 명령이 없다는 조건에 업무 처리도 없고 거절로 닫힌다는 결과를 더했어요. 하지 않아야 할 일도 기대 결과로 적는다는 점을 보세요.

Gherkin은 BDD에서 널리 쓰이는 표현 방법입니다. 이 서식을 썼다고 바로 자동 시험이 완성되는 것은 아닙니다. 실행과 대조를 맡을 코드에 연결해야 해요. 여기서는 문장의 구조를 읽고, 이어지는 HYD의 파이썬 시험에서 같은 전제와 행동, 결과가 어떻게 연결되는지 보겠습니다.

[SLIDE 56] Then의 조건: 실행 결과로 가릴 수 있는 사실

Then을 어떻게 적느냐에 따라 시나리오가 쓸모 있느냐 없느냐가 갈립니다. 화면 왼쪽과 오른쪽을 비교해 보세요.

왼쪽은 거절을 적절히 처리한다, 설비에 영향을 주지 않는다, 담당자에게 잘 알린다. 틀린 말은 아니죠. 그런데 실행 결과를 보고 이게 통과인지 실패인지 정할 수 있을까요? 적절하다는 기준이 읽는 사람마다 다르잖아요.

오른쪽은 어떤가요. 설비 명령이 0건이다, 업무 처리가 0건이다, 처리 건이 거절로 닫히고 사유가 남아 있다. 건수를 세거나 상태와 값을 읽으면 바로 가려집니다.

명세의 수용 기준을 적을 때도 판정할 수 있는 문장으로 적었었죠. Then도 똑같습니다.「영향을 주지 않는다」라고 쓰고 싶다면, 그 영향이 무엇으로 나타나는지를 찾아서「설비 명령이 0건이다」로 바꿔 적는 거예요.

명령 0건은 우리가 검사한 명령 생성 경로에 관한 결과입니다. 설비의 모든 물리적 영향이 없었다는 뜻으로 넓혀 읽지는 마세요. 이 시나리오에서 확인할 업무 처리와 발송, 상태 기록의 범위를 함께 정해야 합니다.

[SLIDE 57] SDD와 BDD: 정하는 일과 확인하는 일

명세를 다룬 SDD와 방금 본 BDD를 나란히 놓겠습니다. 왼쪽 SDD는 무엇을 만들고 무엇을 지킬지 명세를 중심으로 정하고, 구현과 검증에도 그 기준을 이어 가는 접근입니다. 오른쪽 BDD는 구체적인 행동 사례를 함께 논의해서 기대를 맞추고, 그 사례를 실행 가능한 검증에 연결하는 접근이에요.

두 방식을 구현 전과 구현 후로 나누면 정확하지 않습니다. 거절하면 명령이 없어야 한다는 시나리오는 구현하기 전에도 쓸 수 있죠. 오히려 그때 사유가 없으면 거절을 받을 것인지 물어 보면, 명세에서 빠뜨린 조건을 찾을 수 있습니다. 구현한 뒤에는 같은 시나리오로 실제 행동을 확인하고요.

이 수업에서는 앞부분에 명세를 정리했고, 이번 회차에 시나리오와 검증을 다룹니다. 이것은 학습 순서입니다. 실제 작업에서는 명세와 사례를 오가며 기대를 구체화할 수 있어요. 앞 실습에서 저장한 입력과 조회 자료, 기대 결과가 그 출발점입니다.

지키겠다는 문장이 있으면 확인할 사례를 고를 수 있고, 사례가 구체적이면 문장의 모호함을 발견할 수 있습니다. 이렇게 서로 보완하면서 변경의 기준을 남기는 겁니다.

[SLIDE 58] 명세에 적은 것이 시나리오에 놓이는 자리

그럼 명세의 항목 하나하나가 시나리오의 어디로 가는지 표로 맞춰 볼게요. 왼쪽 칸이 명세에 적은 것, 가운데가 시나리오에서 놓이는 자리, 오른쪽이 쿨러 사건의 예입니다.

첫 줄, 입력 계약은 Given과 When에 넣는 입력값이 됩니다. 쿨러 사건이라면 설비 HYD-01과 쿨러 성능 저하 경보가 입력이죠.

둘째 줄, 출력 계약은 Then으로 가요. 결과에 있어야 하는 항목들, 원인이나 추천이나 판단 번호 같은 다섯 항목이 다 있는지를 Then에서 확인합니다.

셋째 줄 금지 조건도 Then으로 갑니다. 다만 이번엔 일어나지 않아야 하는 일로 들어가요. 승인 전에는 설비 명령이 0건이다, 이렇게요. 그래서 이 줄을 강조해 뒀습니다.

마지막 줄 수용 기준은 조금 다릅니다. Then 한 줄이 아니라, 시나리오 한 편 한 편이 통과하는 걸로 확인돼요. 「거절하면 명령도 업무 처리도 없다」는 기준은 관련 시나리오의 결과로 확인합니다.

이렇게 놓고 보면 시나리오에 적는 내용은 새로 지어내는 게 아니에요. 명세에 이미 적은 약속을 전제, 행동, 기대 결과로 나누고, 기대 결과는 실행 결과로 가릴 수 있는 사실로 바꿔 옮겨 놓는 겁니다. 약속을 이렇게 적어 두면, 실행 결과를 보고 그 약속이 지켜졌는지를 가릴 수 있어요. 처음에 물었던, 약속을 어떻게 적어 둬야 확인할 수 있느냐의 답이 바로 이겁니다.

항목이 있다는 것뿐 아니라 값의 형식과 뜻도 계약에 맞아야 합니다. 한 수용 기준을 여러 상황에서 확인할 수도 있어요. 거절 한 번의 통과가 모든 입력과 예외를 대신하지는 않습니다.

[SLIDE 59] 실행 가능한 검증과 네 종류의 시험

약속을 시나리오로 적는 방법까지 찾았으니, 여덟 번째 단원에서는 그 시나리오를 실행 가능한 검증으로 바꾸고, 시험을 네 종류로 나눠 봅니다. Given, When, Then으로 적은 시나리오도 아직은 글이에요. 글로만 두면 결국 사람이 읽고 손으로 확인해야 하죠. 바꿀 때마다 다시 눌러 봐야 하는 반복의 부담이 그대로 남는 겁니다.

그래서 물음이 두 개예요. 시나리오를 어떻게 바꿔야 실행으로 되풀이할 수 있을까요? 그리고 어떤 관점으로 경우를 골라야 중요한 상황을 덜 놓칠까요? 첫 번째 물음부터 풀어 보겠습니다.

[SLIDE 60] 시나리오에서 실행 가능한 검증으로

시나리오를 글로 적었으면 실제 시스템과 연결해야 합니다. 그림 왼쪽의 세 줄이 오른쪽에서 각각 무엇이 되는지 보세요. Given은 시험을 시작할 상태와 입력을 준비하는 코드가 됩니다. When은 그 상태에서 거절 같은 행동을 실행하는 코드가 되고, Then은 나온 결과가 기대 조건을 만족하는지 검사하는 코드가 됩니다.

문장을 적는 것만으로 시스템이 자동 실행되지는 않습니다. 사람이 확인할 수도 있고, 반복할 부분을 시험 코드나 실행 도구에 연결할 수도 있어요. Gherkin을 사용하는 도구에서는 문장에 대응하

는 실행 코드를 연결합니다. 다음 장의 HYD 예는 같은 구조를 파이썬 시험으로 직접 작성한 것입니다.

반복해서 비교하려면 입력뿐 아니라 조회 자료와 규칙, 설정도 확인해야 합니다. 같은 경보를 넣어도 센서 값이 바뀌면 다른 결과가 올바를 수 있잖아요. 저장한 자료로 조건을 고정하면 코드 변경의 영향을 비교하기 쉬워집니다. 실시간 자료를 쓰는 검증이라면 그때 읽은 값과 시각을 함께 남겨야 하고요.

HYD의 규칙 판단 경로는 같은 사실과 같은 규칙에서 같은 판정을 내도록 구성돼 있습니다. 그렇다고 자료를 고정하면 모든 AI의 설명까지 같아진다는 뜻은 아니에요. 오른쪽 대조에서는 그 상황에 맞는 값이나 허용 범위, 일어나지 않아야 할 행동을 검사합니다. 어긋나면 어떤 시나리오의 어느 조건이 실패했는지 남겨 다음 수정으로 이어 갑니다.

[SLIDE 61] 단언: 기대값과 실제 결과를 대어 보는 한 줄

이건 HYD에서 승인 단계를 확인하는 실제 시험 코드 한 편입니다. 코드를 다 읽을 필요는 없어요. 위에서부터 세 군데만 짚을게요.

맨 위의 긴 이름, 이게 시험의 이름입니다. 풀어서 읽으면「거절은 설비와 업무에 아무 일도 하지 않고, 처리 건을 거절로 닫는다」예요. 시험 이름이 그대로 시나리오 문장인 거죠.

가운데 거절을 부르는 줄이 When입니다. 사유로「이번 주 생산 일정상 진행하지 않음」을 넣고 거절을 실행하고요.

아래쪽 assert 두 줄이 Then이에요. assert는 단언이라고 하는데, 뒤에 적은 식이 참인지 확인하는 한 줄입니다. 첫 줄은 명령 번호가 없고 업무 처리 목록이 비어 있는지를 보고요. 둘째 줄은 승인 값이 거절을 뜻하는「반려」인지, 사유가 그대로 남았는지를 봅니다.

화면의 줄임표는 준비 부분하고 나머지 단언을 줄여 놓은 자리예요. 실제 시험엔 메일이 나가지 않았는지 같은 조건도 같이 들어 있습니다. 그리고 단언이 하나라도 참이 아니면, 이 시험은 실패로 기록돼요. 시나리오의 기대가 시험 코드의 검사 조건으로 연결된 겁니다.

이 시험은 준비한 상태와 실행 대역을 사용합니다. 실제 공장에 명령을 보내 본 결과와는 구별해야 해요. 또한 기대 결과 한 줄이 반드시 단언 한 줄과 일대일로 대응하는 것은 아닙니다. 필요한 조건을 빠뜨리지 않고 확인하는지가 중요합니다.

[SLIDE 62] 시험 이름으로 읽는 시나리오

시험 이름을 보면 무엇을 확인하려는지 알 수 있습니다. 영어 이름을 그대로 읽을 필요는 없어요. 왼쪽 상황과 오른쪽 기대 결과를 연결해 보세요.

첫 줄은 권한 있는 승인을 받아 정해진 실행 조건에서 끝까지 처리하는 경우입니다. 둘째는 사유를 적고 거절했을 때 설비 명령과 업무 처리가 없고 거절로 끝나는 경우예요. 셋째는 사유 없는 거절을 받지 않는 경우, 넷째는 지연 알림 뒤에도 승인할 수 있는 경우입니다.

저장소의 승인과 거절 시험은 냉각 대응, 정기 정비, 예비품 구매라는 세 흐름에 같은 시험을 적용합니다. 세 업무가 같은 승인 단계를 공유하기 때문이에요. 그렇다고 모든 시험이 세 흐름에서 똑같이 실행된다는 뜻은 아닙니다. 각 업무의 실제 조치와 별도 조건도 확인해야 하죠.

실패한 시험 이름은 조사할 출발점을 알려 줍니다. 이름만으로 원인이 확정되는 것은 아니므로, 그 시험의 입력과 실패한 단언을 함께 읽어야 합니다.

[SLIDE 63] 한 시나리오가 확인하는 것은 한 경우이다

승인 한 경우가 통과했다고 승인 단계 전체가 확인된 것은 아닙니다. 그림은 같은 출발점에서 갈리는 네 가지 대표 상황이에요.

승인하면 현재 실행 조건도 확인한 뒤 해당 업무가 진행됩니다. 거절하면 설비 명령과 업무 조치 없이 처리 건을 거절로 닫습니다. 응답이 없으면 지연 알림 뒤에도 기다리고, 사유나 권한이 부족한 요청은 받아들이지 않습니다.

이 중 하나만 실행하면 나머지 갈래는 확인하지 못하죠. 그래서 각 행동의 입력과 기대를 준비합니다. 하나의 시험 틀에 여러 입력을 넣어 확인할 수도 있지만, 어떤 경우를 실제로 실행했는지는 구별해서 남겨야 합니다.

이 네 갈래가 가능한 상황을 전부 열거한 것은 아닙니다. 필요한 경우를 빠뜨리지 않으려면 기준값 근처와 자료 누락, 변경의 영향까지 다른 관점으로 살펴봐야 해요. 다음 표의 네 종류가 그 관점을 제공합니다.

[SLIDE 64] 네 종류의 시험: 정상, 경계, 예외, 회귀

중요한 갈래를 고르는 데 쓸 만한 구분이 있습니다. 시험을 확인하는 대상에 따라 정상, 경계, 예외, 회귀, 이렇게 네 종류로 나누는 거예요. 화면 표의 왼쪽 칸입니다.

첫 줄 정상 시험은 기본 경로가 끝까지 가는지를 봅니다. 오른쪽 칸의 예처럼, 승인과 실행 조건을 충족하면 처리 건이 끝까지 처리되는 거죠.

경계 시험은 기준값 바로 안팎에서 판정이 맞는지를 봐요. 쿨러 사건에서는 조치 뒤에 유온을 다시 읽어서 회복했는지를 판정하는데, 이 실습 설비에 정해 둔 회복 기준이 55 ℃ 미만입니다. 그러니 다시 읽은 유온이 딱 55.0 ℃라면 회복으로 보지 않아야 하죠. 이게 경계 시험의 예입니다.

예외 시험은 거절, 누락, 오류처럼 일이 정상 경로를 벗어났을 때 정한 대로 다루는지를 봅니다. 사유 없는 거절이 받아들여지지 않는 게 예외 시험이에요.

마지막 회귀 시험은 무언가를 바꾼 뒤에도 기존 행동이 그대로인지를 봅니다. 긴급 조건을 더한 뒤에도, 거절하면 명령이 0건이어야 하죠.

그런데 넷이 같은 결의 구분은 아니에요. 앞의 셋은 무엇을 확인하느냐에 따른 구분이고, 회귀는 언제 확인하느냐에 가깝습니다. 그래서 회귀 시험의 내용은 대개 이미 있던 정상, 경계, 예외 시험이에요. 정상 시험은 앞에서 본 승인 시나리오가 바로 그 예니까, 나머지 셋을 차례로 보겠습니다.

[SLIDE 65] 경계값 분석: 기준값의 바로 안쪽과 바깥쪽

경계 시험은 판정이 갈리는 지점을 집중해서 확인합니다. 그림의 회복 기준은 유온 55도 미만이에요. 55도보다 낮아야 충족하고, 정확히 55도이면 충족하지 않습니다.

왼쪽의 실제 사건 값은 약 52도였죠. 그 값은 회복 구간 안에 있습니다. 하지만 그 한 번으로 기준값 바로 옆에서도 판정이 맞는지는 알 수 없어요. 미만을 이하로 잘못 구현했어도, 구간 안쪽의 값에서는 같은 결과가 나오니까요.

그래서 여기서는 소수 첫째 자리까지 값을 넣는 예로, 54.9도와 정확히 55도, 55.1도를 비교합니다. 첫 값만 회복 기준을 충족해야 해요. 기준 바로 아래와 기준 자체를 비교하면 등호를 잘못 넣은 오류를 찾을 수 있고, 위쪽 값으로 반대 구간도 확인할 수 있습니다.

0.1도 간격이 모든 센서의 공통 기준은 아닙니다. 실제 자료의 정밀도와 시스템이 값을 다루는 단위에 맞춰 시험 값을 고릅니다. 예상과 다른 결과가 나오면 비교 연산뿐 아니라 단위 변환이나 반올림, 읽은 값이 맞는지도 살펴봐야 해요. 기준에서 먼 값 하나보다 경계를 나누어 보는 이유입니다.

[SLIDE 66] 시간의 경계: 승인 지연 시간의 직전과 직후

숫자의 경계는 시간에도 있습니다. HYD의 이 승인 흐름은 업무상 10분을 지연 알림 기준으로 씁니다. 화면은 그 직전, 기준을 지난 뒤, 알림 뒤의 대기, 뒤늦은 승인을 나눈 것입니다.

먼저 기한 직전에는 지연 알림이 없어야 합니다. 기한에 도달해 확인하는 시점에는 알림 대상이 되고요. 자동 시험에서는 시간을 진행시켜 기한 전후의 상태를 확인할 수 있습니다. 실제로 10분 동안 기다려야만 시험할 수 있는 것은 아니에요. 수업 환경의 20배속에서는 업무상 10분이 실제 30초에 해당합니다.

다음은 알림 이후입니다. HYD의 이 정책에서는 응답이 늦었다고 자동 거절하거나 승인하지 않습니다. 지연을 알리고 처리 건을 대기 상태로 유지해요. 저장소 시험에는 훨씬 뒤에도 처리 건이 계속 대기하는지 확인하는 경우가 있습니다.

마지막으로 뒤늦은 승인도 접수할 수 있어야 합니다. 다만 승인을 받았다는 것과 조치를 실행할 수 있다는 것은 다르죠. 현재 조건 재검사도 통과해야 실행으로 이어집니다. 다른 업무에서는 기한 뒤 자동 종료 정책을 둘 수도 있지만, 이 시험이 확인하는 기준은 HYD에 정해 둔 대기 정책입니다.

[SLIDE 67] 예외 시험: 잘못되었을 때 정한 대로 행동하는가

예외 시험은 잘못된 입력이나 실패가 생겼을 때 정한 대로 대응하는지 봅니다. 화면의 네 카드는 HYD에서 살펴볼 대표 상황입니다.

도구 오류가 나면 오류를 기록하고, 읽지 못한 값을 만들어 넣지 않아야 합니다. 근거가 오래됐다면 현재 판단에 쓸 수 있는지 확인하고 부족하면 보류해야 해요. 여기의 60초는 HYD 사례의 신선도 기준입니다. 방금 조회했다는 이유만으로 오래된 관측값이 최신 값이 되지는 않습니다.

필수 정보가 없으면 무엇이 부족한지 구별합니다. 사람의 답으로 보완하는 흐름에서는 질문하고, 근거 부족으로 보류하는 흐름에서는 사유를 남깁니다. 질문을 했다고 승인 권한까지 생기는 것은 아니죠.

권한 없는 요청은 거부돼야 합니다. HYD는 실제 역할 구성원인지와 지정된 승인 역할 또는 더 높은 등급인지 함께 확인합니다. 역할 이름을 직접 적거나 같은 등급이라는 이유만으로 다른 역할의 권한을 얻지는 않습니다.

여기서 기대한 오류나 보류가 나왔다면 시험은 통과할 수 있어요. 업무가 정상 완료되지 않았다는 것과 시험이 실패했다는 것은 다릅니다. 시험은 그 상황에서 약속한 대응이 나왔는지를 확인하니까요.

[SLIDE 68] 회귀 시험: 변경 뒤에 다시 실행하는 기존 시나리오

마지막이 회귀 시험입니다. 무언가를 바꾼 뒤에 기존 시나리오를 다시 실행하는 시험이에요.

화면 왼쪽이 변경 전입니다. 앞에서 본 승인 단계의 네 갈래, 승인, 거절, 무응답, 사유 없는 요청의 시나리오 넷이 모두 통과한 상태죠. 가운데 화살표에서 요구가 바뀌어요. 명세에 긴급 조건이 하나 더해집니다.

오른쪽 변경 후를 보면, 맨 아래에 새 시나리오가 하나 늘었죠. 긴급 조건을 확인하는 시나리오입니다. 이때 새 시나리오만 돌려 보고 통과했다고 끝내면 어떨까요? 긴급 조건이 잘 동작하는 건 알지만, 거절이나 무응답의 행동이 그대로인지는 모릅니다.

그래서 파란 괄호처럼 기존 넷도 전부 다시 실행해요. 이게 회귀 시험입니다. 회귀라는 말은 되돌아간다는 뜻이에요. 변경으로 기존 행동에 의도하지 않은 영향이 생겼는지 찾는 시험입니다. 바꾼 곳만 확인해서는 이걸 잡을 수 없어요.

[SLIDE 69] HYD의 회귀 시험 기록

화면은 저장소에 남아 있는 서로 다른 시점의 검증 기록입니다. 이번 교재 수정에서 이 시험 전체를 새로 실행했다는 뜻은 아니에요. 기록을 읽는 방법을 보겠습니다.

왼쪽 전체 시험은 2,673개 통과, 16개 건너뜀, 실패 0건으로 적혀 있습니다. 건너뛴 항목은 통과한 항목과 구별해야 하죠. 가운데 쿨러 사건 기록은 경보가 처리 건을 열고, 권한 없는 승인이 거부되고, 승인 뒤 명령과 재관측, 후속 처리가 이어지는지 등 40항목을 확인했습니다. 오른쪽 펌프와 팬 사건은 별도 기록에서 43항목을 확인했습니다.

숫자는 그때 실행한 범위의 요약입니다. 통과 수가 줄었다고 곧바로 기능이 깨졌다는 뜻은 아니에요. 시험 목록과 환경이 달라졌거나 건너뛴 항목이 늘었을 수도 있습니다. 실행 시점과 버전, 실패하거나 건너뛴 이유를 함께 확인해야 합니다.

기존 시험을 남겨 두는 가치는 다음 변경에서 같은 기대를 다시 확인할 수 있다는 데 있습니다. 다만 지금의 요구와 환경에 맞는지도 관리해야 하고, 시험에 없던 상황까지 모두 확인했다고 말할 수는 없습니다.

[SLIDE 70] 에이전트 출력의 반복 검증

아홉 번째 단원은 에이전트 출력의 반복 검증입니다. 앞에서는 거절하면 명령이 없는지처럼 정해진 조건을 확인했습니다. 에이전트에서도 같은 원칙을 쓰지만, 설명 문장이나 실행 과정은 매번 같지 않을 수 있어요.

그렇다면 무엇을 같은 것으로 비교하고, 어떤 차이를 실패로 세어야 할까요? 실제 기록을 먼저 보고 기준을 나눈 뒤, 여러 실행을 함께 해석하는 방법으로 이어가겠습니다.

[SLIDE 71] 같은 종류의 경보를 처리한 두 실행의 비교

이번에는 같은 종류의 쿨러 경보를 처리한 두 기록을 비교해 보겠습니다. 왼쪽은 두 기록에서 같았던 내용이에요. 원인은 쿨러 핀 오염, 추천은 팬 최대 운전이었고, 조치 후보의 평가 점수도 같았습니다.

오른쪽은 달랐던 부분입니다. 진행을 설명한 문장과 도구 호출의 흐름, 걸린 시간이 달랐어요. 한 기록에는 업무 자료 조회와 조치 효과 예측이 나타납니다. 다만 두 경보는 서로 다른 처리 건이고, 한쪽 기록의 전체 도구 호출을 모두 대조한 것은 아닙니다. 모든 입력을 고정한 반복 실험이라고 읽으면 안 돼요.

여기서 구별할 것은 판단 도구의 계산과 에이전트가 일을 진행하는 과정입니다. HYD의 규칙 판단 도구는 같은 사실과 같은 규칙을 주면 같은 판정을 내는 결정론적 경로를 제공합니다. 에이전트는 자료를 찾아 설명하는 과정에서 다른 선택을 할 수 있고요. 실시간 값이 달라지면 규칙 도구의 입력도 달라집니다.

따라서 두 기록의 차이를 모두 모델의 무작위성 때문이라고 결론 내리지는 않습니다. 대신 이 비교를 출발점으로 삼아, 다음 장에서 무엇이 달라도 되는지와 무엇은 반드시 지켜야 하는지를 정해 보겠습니다.

[SLIDE 72] 표현 차이와 계약 위반

시험을 만들기 전에 달라도 되는 것과 지켜야 하는 것을 가릅니다. 화면 왼쪽의 설명 문장은 같은 사실을 다른 낱말로 전달할 수 있어요. 글자가 다르다는 이유만으로 실패로 세면, 의미가 맞는 결과까지 탈락합니다. 대신 근거와 결론이 맞는지는 확인해야 하죠.

도구 순서와 걸린 시간은 조금 더 주의해서 봅니다. 서로 독립적인 조회의 순서는 바뀔 수 있지만, 승인 전에 실행하거나 오래된 값을 최신 값처럼 쓰면 안 됩니다. 처리 시간에 기준이 있다면 그 기준도 지켜야 하고요. 과정의 차이는 필요한 의존 관계와 권한, 시간 조건을 충족하는 범위에서 허용합니다.

오른쪽은 계약 위반입니다. 필수 결과 항목이 빠지거나, 승인 전에 설비 명령과 후속 업무 조치를 실행하거나, 요구한 근거가 없거나, 금지한 조치를 추천하는 경우예요. 이런 위반은 한 번이라도 발견되면 따로 실패로 남깁니다. 판단을 제출하거나 진행 기록을 남기는 모든 쓰기를 승인 위반으로 세는 것은 아닙니다. 금지한 행동의 범위를 정확히 적어야 해요.

너무 엄격하게 글자를 대조하면 의미 있는 차이를 놓치고, 너무 느슨하게 비슷해 보인다고 넘기면 계약 위반을 놓칩니다. 그래서 형식과 의미, 실제 행동의 조건을 나누어 검사합니다. 명세에서 빠진 중요한 조건을 발견했다면 그냥 자유롭게 둔 것이 아니라 기준을 보완할 문제로 다뤄야 합니다.

[SLIDE 73] Then이 견주는 것: 문장이 아니라 계약이 정한 항목

이 구분을 실제 출력에 대어 보겠습니다. 화면의 긴 식별자는 철자를 외우기 위한 것이 아니에요. 어떤 항목을 어떤 방식으로 확인하는지 보시면 됩니다.

첫째는 다섯 필수 항목입니다. 원인과 고장 유형, 진단 가이드 카드, 판단, 판단 번호가 있어야 하고, 약속한 형식과 뜻도 맞아야 합니다. 이름만 있는 빈 값으로는 충분하지 않죠.

둘째는 원인과 추천 조치입니다. 예시에서는 쿨러 핀 오염과 팬 최대 운전을 가리킵니다. 정답표가 하나의 원인을 정했다면 그 값과 대조하고, 여러 조치를 허용했다면 추천이 허용 목록 안에 있는지 확인합니다. 실행마다 같은 글자를 요구할지, 허용한 범위를 검사할지는 기대 행동에 따라 정하는 겁니다.

셋째는 설명입니다. 문장을 통째로 복사했는지를 묻지는 않지만, 선택한 원인과 조치를 근거에 맞게 설명했는지는 봐야 합니다. 인용 번호가 있다고 그 설명까지 자동으로 옳아지는 것은 아니에요.

넷째는 판단 번호입니다. 제출마다 새로 생기므로 이전 번호와 같아야 하는 값은 아닙니다. 번호가 있고 형식이 맞는지에 더해, 실제로 저장된 해당 판단을 가리키는지도 확인할 수 있어야 합니다.

HYD의 실행 중 출력 검사는 필수 항목과 형식을 확인하고, 잘못되면 정해진 횟수 안에서 수정을 요청합니다. 이 형식 검사와 지금 설명한 의미 검증은 범위가 다릅니다. 형식 통과를 판단의 정답으로 읽지 않기 위해 다음 장의 채점을 따로 보는 거예요.

[SLIDE 74] 판단 채점: 정답표와 견주는 네 항목

형식이 맞아도 원인을 잘못 고를 수 있습니다. HYD의 판단 채점은 사람이 작성한 정답표와 결과를 대조해서 그 차이를 확인합니다. 조치 후보의 우선순위를 매기는 평가 점수와도 구별해 주세요. 여기서는 결과가 기대에 얼마나 맞았는지를 채점합니다.

표의 첫 줄은 원인과 고장 유형입니다. 두 값이 모두 기대와 맞아야 이 항목을 충족해요. 둘째 줄은 1순위 추천이 허용한 조치 중 하나인지 봅니다. 셋째 줄은 금지 조치를 확인하는데, 후보에서 제외해야 하는지, 1순위로 추천하지 않으면 되는지를 정답표에서 구별합니다. 두 조건은 범위가 다르죠.

넷째 줄은 요구한 근거 식별자가 결과에 얼마나 들어 있는지입니다. 이것은 근거 포함률이에요. 인용한 자료가 설명을 실제로 뒷받침하는지까지 자동 증명하는 점수는 아닙니다.

오른쪽의 30, 30, 20, 20은 HYD의 기본 가중치입니다. 원인과 고장 유형이 틀리면 첫 항목의 30점을 받지 못하고, 일부 금지 조건이나 근거만 충족하면 해당 항목에는 비율에 따른 점수가 붙습니다. 전체는 100점 만점이지만, 이 숫자가 정답일 확률을 뜻하지는 않아요.

정답표는 처음부터 모든 답이 들어 있는 자료가 아닙니다. 해당 상황의 기대 원인과 허용 조치, 금지 조건과 근거를 사람이 검토해 넣어야 합니다. 설명 문장의 품질은 이 네 항목 점수에 직접 들어가지 않으므로 별도로 검토합니다. 채점이 확인하는 범위를 알고 써야 점수가 높은데도 잘못된 설명을 놓치는 일을 줄일 수 있습니다.

[SLIDE 75] 반복 실행: 한 번의 통과로는 모자란 까닭

한 번 좋은 결과가 나와도 다른 실행에서도 같은 기준을 지키는지는 남아 있습니다. 같은 평가 항목을 여러 번 실행하고, 화면의 평균과 최저, 결과의 반복 비율을 함께 보겠습니다.

평균은 이번에 관찰한 실행들의 전반적인 점수입니다. 최저는 그중 가장 낮았던 점수예요. 평균이 높아도 한 번 크게 틀렸다면 최저에서 드러납니다. 다만 관찰한 최저가 앞으로 가능한 모든 경우의 최악을 뜻하지는 않습니다.

세 번째는 HYD 화면에서 흔들림이라고 부르는 값입니다. 코드에서는 원인과 추천 조치의 조합 중 가장 자주 나온 조합의 비율을 계산합니다. 값이 클수록 그 조합이 일관됐다는 뜻이에요. 100퍼센트여도 전체 설명과 모든 근거가 같다는 뜻은 아니고, 같은 오답을 반복했을 수도 있습니다. 일관성과 정답 여부를 함께 봐야 하는 이유죠.

필수 항목 누락이나 승인 전 조치 실행 같은 위반은 평균 점수에 섞어 넘기지 않습니다. 발생 여부와 횟수, 해당 실행의 기록을 따로 남깁니다. 다른 실행의 높은 점수가 그 위반을 없애 주지는 않으니까요.

이 도구는 반복 횟수를 최대 열 번으로 제한하지만, 열 번이면 충분하다는 일반 기준은 아닙니다. 규칙만으로 같은 조건을 계산하는 경로는 똑같은 입력의 단순 반복보다 경계값과 다른 상황을 확인하는 편이 유용합니다. 모델 경로를 비교할 때도 입력과 조회 자료, 설정을 기록해서 무엇이 바뀌었는지 구별해야 합니다.

[SLIDE 76] 실패한 시험에서 고칠 곳 가리기

실패를 찾았다면 이제 원인을 좁혀야 합니다. 표의 가운데 칸은 정답처럼 고칠 곳을 지정하는 칸이 아니라, 먼저 대조할 자료입니다.

단계를 건너뛰었다면 지침과 호출 기록을 함께 봅니다. 지침이 모호했는지, 필요한 도구를 쓰지 않았는지, 호출했지만 실패했는지에 따라 조치가 달라져요. 기준값이 다르면 규칙과 그 버전, 실제 입력과 단위를 확인합니다.

결과 항목이 빠졌다면 먼저 명세에 그 항목이 요구돼 있는지 봅니다. 이미 요구돼 있다면 출력 구현이나 검사가 놓친 것일 수 있어요. 누락됐다는 이유만으로 명세에서 그 항목을 지우는 것은 해결이 아닙니다.

반대로 시험의 기대값을 옮겨 적다가 잘못 적었을 수도 있습니다. 그때는 원래 요구와 근거를 대조해서 시험을 바로잡고 이유를 남깁니다. 요구가 바뀌었다면 명세와 시나리오를 함께 갱신해야 하고요.

마지막 줄이 가리키는 것은 근거 없이 기대값을 낮추는 일입니다. 현재 나온 결과를 통과시키려고 기준을 바꾸면 같은 오류가 다음에도 통과합니다. 실패 기록에서 입력, 기대, 실제 결과를 확인한 다음 원인을 고치고, 해당 시험과 기존 시험을 다시 실행하겠습니다.

[SLIDE 77~78] 실습: 기존 기능과 추가 기능을 함께 검증하기

앞 실습의 프로젝트와 spec.md, 저장한 입력과 기대 결과를 그대로 열겠습니다. 일반 요청, 긴급 요청, 필수 값 누락, 승인 거절을 Given·When·Then으로 적고, 긴급 기준 바로 아래와 정확히 경계인 값도 확인합니다. 기대값은 실행 전에 명세에서 정합니다.

각 시나리오를 실행 가능한 검사로 연결하고 같은 입력과 조건으로 반복해 결과를 비교하겠습니다. 설명의 표현 차이와 출력 계약 위반을 나눠 보고, 실패가 있으면 입력·조회·판단·출력 중 어느 단계에서 달라졌는지 기록으로 좁힙니다. 발견한 원인을 수정한 뒤에는 추가 기능뿐 아니라 기존 시나리오도 모두 다시 확인합니다.

통과 여부, 실제 결과, 수정 사항과 남은 미확인을 저장하겠습니다. 실패가 없으면 일부러 만들지 않습니다. 이 검증이 확인한 조건의 범위도 함께 남기고 전체 제작 흐름을 정리하겠습니다.

[이론 08] 실전반 5회차

[SLIDE 79] 에이전트 제작부터 검증까지의 전체 과정

이번 시간에는 승인·질문 기록과 명세, 시나리오와 실행 기록, 수정 내역를 바탕으로 전체 과정을 정리합니다. 실패가 발견됐다면 어떻게 고쳤는지, 아직 확인하지 못한 조건은 무엇인지도 함께 떠올려 보세요.

우리는 목표와 도구로 에이전트의 일을 구성하고, 스킬과 연결 규약으로 수행 방법을 정리했습니다. 사람의 개입과 명세를 통해 지킬 기준을 세웠고, 이번에는 그 기준을 구체적인 상황에서 검증했습니다.

이제 쿨러 사건 하나에 그 역할을 차례로 놓아 보겠습니다. 새 기능을 더 배우기보다, 어느 장치가 무엇을 확인하고 그 결과가 다음 일로 어떻게 이어지는지 정리하는 시간입니다.

[SLIDE 80] 여섯 개념이 차례로 푼 문제

여섯 개념을 각각 어떤 문제에 답했는지로 묶어 보겠습니다. 그림의 왼쪽은 일을 수행하는 구조, 오른쪽은 기대와 권한을 확인하는 구조입니다. 이 과정에서 우리가 함께 사용한 개념들이에요.

에이전트는 목표에 맞춰 도구를 선택하고 결과를 보며 다음 행동을 이어 갑니다. 스킬은 반복해서 쓸 절차와 판단 지침을 묶어 제공합니다. 지침을 적었다고 실행 순서가 항상 강제되는 것은 아니므로 실제 호출 기록도 확인해야 하죠. MCP는 도구와 자료를 연결하는 공통 규약이고, HYD는 이를 이용해 업무 시스템을 연결했습니다.

HITL은 필요한 정보나 결정을 사람에게 받는 자리를 둡니다. SDD는 명세를 구현과 변경의 기준으로 유지하고, BDD는 구체적인 행동 사례로 기대를 맞추고 검증에 연결합니다.

이 여섯 이름을 모든 에이전트가 반드시 같은 순서로 갖춰야 하는 목록으로 외우지는 마세요. 업무에 따라 도구를 연결하는 방식과 검토 절차는 달라질 수 있습니다. 중요한 것은 일을 수행할 수 있는가에 더해, 무엇을 지켜야 하고 누가 결정하며 어떻게 확인할 것인가를 함께 설계하는 일입니다.

[SLIDE 81] 쿨러 사건 한 건 위에 놓인 장치

같은 개념을 쿨러 사건의 흐름에 놓아 보겠습니다. 왼쪽에서 경보가 열리고, 에이전트가 스킬을 읽어 도구를 호출하고, 추천안을 제출합니다. 오른쪽에서는 검사와 담당자의 승인, 현재 조건 확인을 거쳐 설비 명령으로 이어집니다.

기록의 경보 시각은 11시 13분 8초, 승인 시각은 11시 14분 56초였습니다. 이 두 시점 사이는 약 2분이에요. 전체 복구 시간이나 에이전트 혼자 일한 시간을 뜻하는 것은 아닙니다.

도구 호출 과정에서는 대안 평가 응답이 커서, 필요한 항목으로 좁혀 보겠다는 계획도 남겼습니다. 앞에서 확인한 기록만으로 실제 재조회가 성공해 모든 항목을 읽었다고까지 결론 내리지는 않았죠. 계획과 수행 기록을 나눠 읽어야 근거가 끊긴 곳을 찾을 수 있습니다.

추천안의 자동 검사는 정해진 조건을 확인하고, 담당자의 승인은 실행에 대한 결정을 남깁니다. 두 자리를 통과해도 승인 시점의 설비 조건을 다시 확인해야 합니다. 이 그림은 각 역할을 묶은 것이고, 실제 처리에는 그 사이의 검사들이 함께 있습니다.

아래 시나리오와 회귀 시험은 이 흐름을 정한 입력과 조건에서 다시 확인합니다. 사건 한 건의 성공 기록은 그 사례의 근거이고, 여러 조건을 시험하는 일은 별도의 검증입니다. 둘을 함께 봐야 같은 방식으로 맡길 수 있는 범위를 판단할 수 있어요.

[SLIDE 82] 목표, 도구, 멈춤 조건: 하나가 빠졌을 때

이번에는 목표와 도구, 멈춤 조건이 각각 어떤 빈칸을 메우는지 보겠습니다. 없으면 반드시 어떤 오답이 나온다는 예언이 아니라, 설계할 때 확인할 질문입니다.

목표에는 무엇을 내면 일이 끝나는지가 들어갑니다. 쿨러 사건이라면 단순히 설명을 잘하는 것을 넘어, 근거 있는 추천안을 약속한 형태로 제출하는 것이죠. 끝의 기준이 모호하면 불필요한 조회를 이어 가거나 필요한 일을 덜 한 상태에서 끝날 수 있습니다.

도구는 현재 자료를 확인할 통로입니다. 입력으로 받은 자료만으로 충분한 업무도 있지만, 지금의 유온이나 납기 상황이 필요한 업무라면 그 자료를 얻을 경로가 있어야 해요. 도구가 있어도 필요한 값을 실제로 읽었는지 확인해야 합니다.

멈춤 조건은 근거 부족과 도구 실패를 어떻게 다룰지 정합니다. 확인하지 못한 값을 추정해 결과를 완성하지 않고, 질문하거나 보류하고 사유를 남기는 조건이에요. 중단 기록을 보면 업무를 못 했다는 사실뿐 아니라 무엇이 부족했는지도 알 수 있습니다.

목표는 끝을, 도구는 근거의 통로를, 멈춤 조건은 확인하지 못했을 때의 행동을 정합니다. 이 세 가지가 이어져야 성공한 결과와 보류한 결과 모두를 해석할 수 있습니다.

[SLIDE 83] 근거가 놓인 자리: 지식과 현재값

에이전트가 추천안을 낼 때 쓴 근거는 어디서 왔을까요? 화면에 두 갈래로 나눠 놨습니다.

위쪽 회색 구획이 지식이에요. 원인 후보 셋에서 고장 유형을 거쳐 조치 방법으로 가는 관계죠. 쿨러 핀 오염, 팬 구동 불량, 주변 온도 상승이 쿨러 냉각 성능 상실을 일으키고, 그 고장 유형에 대한 완화 후보 중 하나가 팬 최대 운전입니다. 원인과 현재 조건에 따라 그 후보를 적용할 수 있는지는 따로 가립니다. 이건 지식 그래프에 미리 정해 둔 관계입니다.

아래쪽 하늘색 구획은 현재값이에요. 냉각 효율이 절반 아래로 떨어졌다는 값은 시계열 DB에서, 납기가 걸린 생산 오더는 업무 DB에서 읽었습니다. 도구를 통해 판단에 사용한 자료예요. 조회한 시각과 값이 관측된 시각도 함께 확인해야 합니다.

그런데 이 두 갈래는 서로가 필요해요. 지식에는 원인마다 어느 센서 값을 어떤 조건으로 확인할지가 적혀 있습니다. 쿨러 핀 오염이라면 냉각 효율이 낮아졌는지를 보라는 식이죠. 하지만 그 값이 지금 몇인지는 지식에 없어요. 판단하는 순간 시계열 DB에서 읽어 와야, 이 원인이 지금 사건에 맞는지 가릴 수 있습니다. 그래서 두 갈래가 가운데 도구 호출에서 모여서, 오른쪽 추천안이 되는 거예요.

지식의 규칙과 관계도 개정될 수 있고, 관측값은 시간에 따라 달라집니다. 적용한 지식 버전과 관측 시점을 함께 남겨야 해요. 둘을 섞지 않는 것이 근거를 다루는 기준이었던 까닭이 여기 있습니다. 추천안에는 근거마다 어디서 읽었는지가 남아 있어요. 덕분에 나중에 결과가 이상하면, 어느 갈래에서 틀렸는지 거슬러 갈 수 있습니다.

[SLIDE 84] 스킬과 MCP: 바꾸려는 것에 따라 고칠 곳

일하는 방법을 바꾸는 것과 연결할 수 있는 범위를 바꾸는 것은 고칠 자리가 다릅니다. 왼쪽 스킬은 어떤 근거부터 확인하고 부족할 때 어떻게 대응할지 같은 지침을 담습니다. 오른쪽 도구 연결은 어느 시스템의 어떤 기능을 사용할 수 있는지와 관련돼요.

예를 들어 같은 자료를 읽되 확인 순서를 개선하려면 스킬을 검토할 수 있습니다. 새 업무 시스템의 자료가 필요하다면 도구 제공과 연결, 접근 권한을 확인해야 하죠. 호출은 됐는데 권한 오류가 났다면 지침 문장을 길게 쓰는 것으로 해결되지는 않습니다.

출력 항목을 바꾸는 경우에는 스킬만 고쳐서도 안 됩니다. 명세의 계약과 결과를 읽는 프로그램, 출력 검사를 함께 확인해야 합니다. 스킬이 약속을 설명하는 자리라면 실제 검사는 구현이 맡기 때문이에요.

MCP는 연결 규약입니다. 그 이름 자체가 모든 권한을 통제하거나 실행 환경 전체를 격리한다는 뜻은 아닙니다. 실제 허용 범위는 호스트와 서버의 설정, 계정 권한과 도구 구현이 결정합니다. 그래서 원하는 일을 못 했을 때도 지침과 연결, 실제 응답을 구별해서 살펴보는 겁니다.

[SLIDE 85] 검사를 통과한 것과 승인된 것

자동 검사를 통과한 것과 사람이 승인한 것을 구별해 보겠습니다. 가운데의 자동 검사는 HYD 추천안의 형식과 규칙 인용, 승인 역할 같은 조건을 확인합니다. 통과는 정해진 검사에서 위반이 발견되지 않았다는 뜻이에요.

오른쪽 담당자의 승인은 그 안을 실행하도록 허락한 결정입니다. 승인 권한을 확인한 경로에서 사람과 역할, 선택한 안과 시각을 기록합니다. 에이전트가 승인자 이름을 결과에 써 넣었다고 이 기록이 만들어지는 것은 아닙니다.

앞 이론의 다른 AI가 초안을 검토한 것도 떠올려 보세요. 그 검토는 참고할 의견입니다. 자동 검사 통과나 사람의 승인을 대신하지 않아요. 여러 AI가 같은 답을 했더라도 근거와 권한을 따로 확인해야 합니다.

그림 아래의 질문과 작업 닫기는 또 다른 사람의 개입입니다. 부족한 정보를 답하는 것과 실행을 허락하는 것은 역할이 다르죠. 승인 뒤에도 현재 조건을 다시 검사하고, 명령을 보낸 뒤에는 실행 결과를 확인해야 합니다. 그림의 실행 허락을 실제 조치의 성공과 같은 뜻으로 읽지 않는 것이 중요합니다.

[SLIDE 86] 명세, 시나리오, 가드레일, 승인이 추천안에 걸리는 자리

명세와 시나리오, 가드레일과 승인은 각각 어디에 걸릴까요? 한 그림에 이어 봤습니다. 왼쪽 위 명세에서 출발할게요.

명세에서 화살표가 두 개 나가죠. 하나는 가운데 에이전트로 가서, 에이전트가 지킬 기준이 됩니다. 다른 하나는 아래 시나리오로 가서 기대값이 되고요. 같은 명세가 만드는 쪽에는 기준이, 확인하는 쪽에는 기대값이 되는 겁니다. 출력 계약으로 예를 들어 볼게요. 원인부터 판단 번호까지 다섯 항목을 낸다는 약속은, 에이전트 쪽에서는 작업에 붙은 결과 형식이 되고, 시나리오 쪽에서는 「다섯 항목이 있고 원인과 추천이 이 식별자다」라는 Then이 됩니다.

왼쪽 아래 시나리오에서는 에이전트로 화살표가 가요. 에이전트의 행동과 그 결과를 이어 받는 업무 흐름을 확인하는 거죠.

에이전트는 가운데 추천안을 냅니다. 그 추천안에 오른쪽 위의 가드레일이 검사를 걸고, 오른쪽 아래의 담당자가 승인을 걸어요.

여기서 화살표가 닿는 곳을 보세요. 시나리오는 에이전트와 연결된 업무 흐름을 검증하고, 가드레일과 승인은 그 에이전트가 낸 결과 하나하나에 닿습니다.

[SLIDE 87] 요구가 바뀔 때의 순서

명세와 시나리오와 회귀는 변경 한 번 안에서 어떤 순서로 쓰일까요? 화면에 네 단계로 이어 놨습니다. 단계마다 아래 줄은 앞의 실습에서 다룬 긴급 정비 조건의 예예요.

요구가 바뀌면 코드부터 고치지 않습니다. 첫 단계는 명세예요. 새 조건의 계약과 금지 조건, 수용 기준을 적습니다. 긴급 조건을 더하더라도, 「승인 없이 확정하지 않는다」는 그대로 남겨 두고요. 왜 이걸 굳이 다시 적을까요? 긴급이라는 조건은 빨리 처리하자는 쪽으로 기울기 쉽죠. 모호하게 덧붙인 기능이 기존 출력이나 승인 규칙을 흔들 수 있다는 건 명세를 배울 때 봤습니다. 지킬 것을 명세에 그대로 남겨 둬야, 나중에 전체를 다시 돌릴 때 그 약속이 아직 지켜지는지를 확인할 기준이 생깁니다.

둘째 단계에서 그 조건의 시나리오를 더합니다. 긴급 요청이 들어왔을 때의 Given, When, Then과 기대값이요.

셋째 단계에 가서야 구현해요. 스킬의 절차나 해당 기능의 코드를 명세에 맞춰 고치죠.

마지막 단계는 전체를 다시 실행하는 겁니다. 새 시나리오만이 아니라, 일반 요청, 필수 정보가 빠진 요청을 확정하지 않는 보류, 거절의 기존 시나리오까지 전부요.

순서가 왜 이렇게 중요할까요? 구현 결과만 보고 기대값을 정하면, 잘못된 행동도 정답으로 옮겨 적을 위험이 있습니다. 구현 뒤에 시험을 보완하더라도 원래 요구와 근거에서 기대를 정해야 해요. 기대값을 먼저 적어 둬야, 구현이 그걸 맞혔는지 확인할 수 있어요.

[SLIDE 88] 실행할 때마다 작동하는 장치와 바꿀 때마다 하는 검증

지금까지 본 장치들은 작동하는 때가 둘로 나뉩니다. 실행할 때마다 작동하는 것, 그리고 바꿀 때마다 하는 것이에요.

화면 왼쪽이 실행할 때마다 작동하는 장치입니다. 출력 계약의 검사, 가드레일, 사람의 승인이죠. 에이전트가 추천안을 낼 때마다 그 한 건을 검사하고 걸러요. 정해 둔 조건을 어긴 결과를 막는 역할입니다. 가능한 모든 잘못을 잡는 것은 아니에요.

오른쪽은 바꿀 때마다 하는 검증이에요. 시나리오의 실행, 판단 채점, 전체 회귀. 스킬과 명세, 코드뿐 아니라 모델이나 지식, 중요한 설정을 바꿨을 때도 관련 검증을 돌립니다. 찾는 건 변경 때문에 달라진 행동이고요.

오른쪽 둘째 줄의 판단 채점은 이때 바꾸기 전과 후의 실행을 나란히 놓고 볼 수 있어요. 항목마다 점수가 어떻게 달라졌는지, 그리고 그 사이에 지식 그래프에서 끊기거나 새로 생긴 관계가 무엇

인지까지 보여 줍니다. 그래서 점수가 떨어졌을 때, 원인 후보가 될 변경을 좁힐 수 있습니다. 동시에 조회 자료도 달라졌다면 지식 변경 때문이라고 단정할 수 없으므로, 비교 조건을 맞춰 다시 확인해야 해요.

여기서 기억해 둘 게 있어요. 한쪽이 있다고 다른 쪽을 빼면 안 됩니다. 승인 단계가 있으니까 회귀 시험은 필요 없다고 해 볼까요? 그러면 변경 때문에 추천의 질이 떨어져도, 그걸 사람이 매번 떠안게 돼요. 반대로 회귀 시험을 다 통과했으니 승인은 필요 없다고 하면요? 시험이 다루지 않은 상황에서, 잘못된 한 건이 그대로 실행됩니다.

[SLIDE 89] HYD의 세 에이전트가 내는 산출물

앞에서 승인과 거절의 시험을 세 흐름에 적용한다고 했죠. 그 세 흐름을 맡은 에이전트가 화면의 세 줄입니다. 같은 방식으로 만든 에이전트라도, 맡은 일이 다르면 내는 산출물이 달라져요.

먼저 가운데 하늘색 구획을 보세요. 셋에 공통인 틀이에요. 목표가 있고, 스킬과 도구가 있고, 출력 계약이 있죠. 틀은 같습니다.

다른 건 그 틀에 넣은 거예요. 맨 위 줄은 쿨러 성능 저하 경보를 맡은 냉각 긴급 대응입니다. 오른쪽 끝을 보면, 팬 최대 운전과 대안 셋을 담은 추천안을 내죠. 가운데 줄은 정기 정비 도래를 맡은 정기 정비 계획이에요. 언제 정비할지를 정한 정비 시간을 냅니다. 맨 아래 줄은 예비품 재고가 기준을 벗어났을 때의 예비품 구매고요. 공급사와 수량과 금액을 담은 발주안을 냅니다.

넣은 게 다르다는 건 스킬만 얘기하는 게 아니에요. 붙은 도구도 다릅니다. 냉각 긴급 대응은 원인을 진단하고 조치 후보를 평가하는 판단 도구를 써요. 정기 정비 계획은 정비 기한과 허용 한계까지 남은 시간, 예정된 야간이나 주말 정비 시간을 읽는 도구를 쓰고요. 예비품 구매는 재고와 공급사 견적을 읽는 도구를 씁니다. 업무가 다르니 읽을 값과 지킬 규칙이 다르고, 그래서 산출물의 항목도 달라지는 겁니다.

그러니까 새 업무를 에이전트에 맡길 때도 틀부터 새로 만들 필요는 없어요. 공통 구조를 활용하되 목표와 스킬, 도구와 권한, 출력 계약을 그 업무에 맞춰 확인해야 합니다. 필요한 연결이나 실행 기능이 없으면 구현도 더해야 하고요. 산출물이 달라지는 건 틀이 달라서가 아니라, 그 자리에 넣은 게 달라서예요.

[SLIDE 90] 실습 산출물과 근거를 다루는 방식

여러 업무에 에이전트를 적용할 때의 산출물 예 다섯을 표로 모았습니다. 이번엔 오른쪽 칸, 근거를 다루는 방식을 중심으로 보세요.

첫 줄 점검 계획은 확인되지 않은 원인을 가설로 표시합니다. 품질 대응 카드는 원인이 정해지지 않았으면 단정하지 않고 조사 요청으로 냅니다. 도표와 이상 구간 요약은 두 값이 함께 움직였다는 것만으로 원인이라고 단정하지 않습니다. 작업 목록은 담당자와 기한이 없으면 미정으로 적고, 학습 코치는 정답에 출처를 붙여서 되짚어 줍니다.

형식은 계획, 카드, 도표, 목록, 피드백으로 다 다릅니다. 그런데 오른쪽 칸엔 공통점이 있어요. 모르는 건 확정하지 않고, 아는 것에는 출처를 붙인다는 겁니다. 산출물의 모양은 응용마다 달라져도, 근거를 다루는 이 방식은 달라지지 않아요.

인수인계서에도 같은 방식을 적용합니다. 한 출처를 읽지 못했으면 빈칸으로 두지 않고, 읽지 못했다고 적습니다. 이 방식이 왜 그렇게 중요할까요? 산출물을 받는 쪽은 사람이든 시스템이든, 적힌 대로 믿고 다음 일을 시작하거든요. 확정하지 않은 걸 확정처럼 적으면, 그 잘못이 다음 일로 그대로 넘어갑니다. 반대로 확인하지 못했다고 적혀 있으면, 받는 쪽이 거기서 멈추고 확인할 수 있죠.

[SLIDE 91] 산출물의 형식을 정하는 것: 읽는 쪽과 이어지는 일

산출물의 형식은 누가 읽고 그 뒤에 어떤 일을 하는지에 맞춰 정합니다. 인수인계서는 다음 담당자가 읽고, 추천안은 승인 화면을 만드는 시스템과 승인할 사람이 함께 읽습니다.

사람에게는 맥락과 근거, 미확인 부분이 잘 보이는 문서와 도표가 도움이 됩니다. 시스템과 주고받을 때는 항목의 이름과 형식, 값의 뜻을 약속해야 해요. HYD는 그 전달에 JSON을 사용하지만, 모든 시스템이 같은 형식을 쓰는 것은 아닙니다.

값에도 조건이 있습니다. 예를 들어 안을 구별하는 번호가 중복되면 사람이 고른 안과 시스템이 처리할 안을 명확히 연결할 수 없죠. HYD의 출력 계약은 안마다 다른 구분 값을 요구합니다. 사람만 읽는 문서라도 이런 모호함을 그대로 넘겨서는 안 됩니다.

추천안은 승인과 실행 검토로, 품질 대응 카드는 조사로, 인수인계서는 다음 교대 작업으로 이어집니다. 그래서 형식 검사에 더해 내용과 근거도 확인해야 합니다. 읽는 쪽이 필요한 결정을 정확히 이어 갈 수 있게 만드는 것이 출력 계약의 목적입니다.

[SLIDE 92] 검증 실습의 산출물이 쓰이는 곳

실습의 산출물은 그 실습에서 끝나지 않고 다음 일로 이어집니다. 화면 왼쪽 회색 구획이 사람의 결정과 명세를 다룬 실습의 산출물이고, 가운데 하늘색 구획이 시나리오와 회귀를 다룬 실습, 그러니까 이번 회차에 해 본 실습의 산출물이에요.

왼쪽 맨 위의 확인 질문과 전후 기록은 필수 정보가 빠진 경우의 시나리오가 됐습니다. 그 아래 spec.md와 기대 결과는 시나리오의 재료가 됐고요. 명세 실습에서 저장해 둔 입력과 기대값이 바로 시나리오의 Given과 Then이 된 거예요.

가운데 시나리오는 실행돼서 회귀 기록으로 남습니다. 그리고 오른쪽 화살표처럼, 그 기록이 다음에 요구가 바뀔 때 다시 실행할 기준이 되죠.

왼쪽 맨 아래의 초안, 검토 의견, 수정본은 에이전트끼리의 검토와 사람의 승인을 서로 다른 항목에 남기는 예입니다. AI의 검토 의견을 자동 검사나 사람의 승인 기록과 혼동하지 않게 구별합니다. 별도 응용 예인 가운데 아래 인수인계서는 근거 시각과 함께 다음 교대의 담당자에게 넘어가고요. 값마다 어디서 언제 읽었는지를 붙이는 건, 근거가 놓인 자리에서 본 현재값의 원칙 그대로입니다.

[SLIDE 93] 운영 중 결과가 기대와 다를 때 먼저 살펴볼 곳

이번엔 에이전트를 운영하다가 결과가 기대와 다를 때를 생각해 볼게요. 증상이 가리키는 장치부터 살펴보면, 고치지 않아도 될 곳을 건드리지 않습니다. 표의 왼쪽 칸이 증상, 가운데가 먼저 살펴볼 곳이에요.

첫 줄, 값이 실제와 다르면 도구와 실행 기록을 봅니다. 호출 기록에서 무엇을 물었고 무엇을 받았는지, 값이 얼마나 묵었는지를 확인해요. 호출 기록부터 보는 데는 까닭이 있습니다. 도구를 부를 때마다 입력과 응답, 시각과 걸린 시간이 한 줄씩 남거든요. 그래서 값이 이상할 때, 도구가 애초에 그 값을 돌려줬는지, 아니면 받은 값과 다르게 옮겨 적었는지가 여기서 갈립니다. 어느 쪽이냐에 따라 고칠 곳이 달라지니까, 짐작으로 고치기 전에 기록부터 보는 거죠.

둘째 줄, 필요한 값을 받지 못했다면 도구를 호출했는지부터 보고, 서버 연결과 권한, 오류 응답을 차례로 살펴봅니다.

셋째 줄, 묻지 않고 확정했다면 금지 조건과 사람의 개입 지점, 그러니까 명세와 HITL 쪽이에요. 마지막 줄, 고친 뒤에 기존 요청의 결과가 달라졌다면 회귀 시험에서 실패한 시나리오를 봅니다.

검증 단원에서 본「실패한 시험에서 고칠 곳」표는 시험이 실패한 데서 출발했죠. 이 표는 운영 중에 본 결과에서 출발합니다. 출발점은 달라도 생각하는 방식은 같아요. 증상을 출발점으로 기록을 대조하고, 확인한 원인에 맞춰 고치는 겁니다.

[SLIDE 94] 사람이 정하는 기준과 에이전트가 정하는 과정

마지막으로 사람이 정하는 기준과 에이전트가 실행 중 선택하는 과정을 나누어 보겠습니다. 왼쪽은 목표와 지침, 도구 권한, 계약과 금지 조건, 시나리오의 기대값입니다. 업무를 맡기기 전에 합의하고 관리할 기준이에요.

오른쪽은 그 기준 안에서 어떤 도구를 어떤 입력으로 부를지, 받은 결과에 따라 무엇을 더 확인할지, 어떻게 설명할지입니다. 서로 독립적인 조회의 순서나 설명의 문장은 달라질 수 있습니다. 하지만 필요한 확인을 건너뛰거나 권한을 넘어서는 선택까지 허용되는 것은 아니죠.

왼쪽 기준도 영원히 바뀌지 않는 것은 아닙니다. 요구가 바뀌면 근거를 검토하고 명세와 검사를 함께 바꾸며, 어느 버전에 어떤 기준을 적용했는지 남깁니다. 실행 결과에 맞춰 아무 설명 없이 기준을 바꾸는 일과는 다릅니다.

앞의 쿨러 기록에서 재조회를 하겠다는 말과 실제로 확인된 호출을 구별한 것도 같은 이유입니다. 계획만으로 수행을 인정하지 않고, 기록이 보여 주는 범위에서 판단하는 거예요.

맡길 수 있는 범위는 멋진 답변 한 번으로 정해지지 않습니다. 기대와 권한을 정하고, 실제 결과와 남은 불확실성을 확인하고, 바꾼 뒤 다시 검사하는 과정을 통해 근거를 쌓아 갑니다. 이제 그 흐름을 한 가지 금지 조건에 대어 복습해 보겠습니다.

[SLIDE 95] 금지 조건 한 줄이 지켜지는 자리

그 겹겹의 장치가 실제로 어떻게 맞물리는지, 복습하면서 확인해 보겠습니다. 이번 회차에 배운 흐름을 「거절하면 설비 명령이 없다」는 금지 조건 한 줄에 대어 볼 거예요. 표의 다섯 줄은 이 한 줄이 놓이는 자리와, 그 자리에서 어떤 모양이 되는지를 보여 줍니다. 먼저 위에서부터 같은 한 줄이 줄마다 어떤 모양이 됐는지 대어 보고, 넷째 줄 회귀와 다섯째 줄 실행 중 장치가 각각 언제 작동하는지 가려 보세요.

위의 세 줄부터 볼게요. 명세에서는 금지 조건과 수용 기준의 문장입니다. 구현하기 전에 정하죠. 시나리오로 오면 Then이 됩니다. 설비 명령 0건, 업무 처리 0건, 거절로 닫힌다. 실행 결과로 가릴 수 있는 사실로 바뀌었죠. 시험에서는 단언이 돼요. 명령 번호가 없고 업무 처리 목록이 비어 있는지를, 시험을 돌릴 때마다 확인합니다.

넷째 줄 회귀는 바꿀 때마다 작동합니다. 긴급 조건을 더한 뒤에 거절 시나리오를 다시 돌려서, 변경 때문에 거절의 행동이 달라지지 않았는지 찾죠.

다섯째 줄은 실행 중이에요. 가드레일 검사, 그리고 담당자의 승인과 거절은 추천안이 나올 때마다 작동합니다. 지금 이 한 건이 잘못 실행되는 걸 막는 거예요. 고른 답이 이렇게 갈렸다면 맞게 가린 겁니다. 같은 한 줄을 서로 다른 때에 지키는 두 장치인 거죠.

[SLIDE 96] 복습: 결정의 자리와 바뀐 요구의 확인

전체 흐름에 이어, 변경 전후 기록과 명세를 바탕으로 사람의 결정과 변경 조건을 돌아보겠습니다. 결정의 자리와 바뀐 요구의 확인을 다섯 상황에 대어 볼 거예요. 표의 왼쪽 상황마다, 가운데 열에서 먼저 무엇을 갈라야 하는지 골라 보세요. 오른쪽 근거는 가린 채로 생각해 보시고요.

첫째, 추천안이 올라온 지 10분이 지나도록 결정이 없습니다. 이건 사람의 대기예요. 에이전트가 멈춘 게 아닙니다. 지연 알림만 나가고, 승인 작업은 열린 채로 담당자를 기다리죠.

둘째, 에이전트가 대상 설비를 묻고 멈췄어요. 업무 정보를 묻는 질문입니다. 답은 빠진 값만 채우고, 승인은 승인 단계에서 따로 받아요.

셋째, 쿨러 사건의 추천안이 가드레일을 통과했습니다. 올려도 된다는 확인일 뿐이에요. 실행해도 되는지는 운전원이 승인하면서 남긴 결정 기록에서 확인합니다. 누가, 어떤 역할로, 승인 순간 어떤 값을 다시 보고 결정했는지가 거기 남아 있죠.

넷째, 긴급 표시를 넣으면서 추천 항목의 이름을 바꾸려 해요. 계약을 깨는 변경이죠. 출력 계약의 이름과 형식과 뜻을 보면 가를 수 있습니다. 기존 이름과 뜻을 유지하며 확장할 수 있는지, 소비자가 추가 항목을 받아들이는지 확인해야 해요.

다섯째, 고친 뒤 긴급 경보 한 건만 실행해 봤습니다. 새 기능만 확인한 거예요. 기존 요청과 긴급 요청, 보류할 요청의 수용 기준을 함께 다시 실행해야, 기존 기능이 지켜졌는지 알 수 있습니다.

다섯 상황을 묶어 보면, 앞의 셋은 누가 결정하고 무엇이 결정이 아닌지를 묻고 있어요. 뒤의 둘은 무엇이 달라져도 되고, 무엇이 그대로여야 하는지를 묻고요.

[SLIDE 97] 오늘 다룬 핵심 개념과 주요 용어 정리

이어서 용어 정리입니다. 이번 회차에 다룬 일곱 개념의 이름과 뜻을 표 한 장에 모았어요. 영문 정식 명칭은 화면에 있으니, 저는 뜻을 짚겠습니다.

HITL은 자동 처리의 정해진 지점에 사람의 결정을 넣는 방식이에요. 가드레일은 입력과 출력, 행동이 정한 경계를 지키도록 검사하는 장치입니다. 이 회차에서는 HYD 추천안의 규칙 검사를 봤고요.

바이브 코딩은 자연어 요청과 실행 결과를 중심으로 AI가 만든 코드를 고쳐 가는 방식으로 다뤘습니다. 여기서 비교한 것은 명세와 체계적인 검토가 생략된 경우예요. SDD는 구현하기 전에 명세를 정하고, 그 명세로 구현하고 확인하는 개발 방식이죠.

명세는 무엇을 받아 무엇을 내고, 무엇을 하지 않는지 적은 문서예요. 계약은 주는 쪽과 받는 쪽이 지키기로 한 항목과 형식, 뜻과 처리 조건이고요. 마지막으로 수용 기준은 구현이 끝났는지를 참과 거짓으로 가를 수 있게 적은 조건입니다.

[SLIDE 98] 오늘 다룬 핵심 개념과 주요 용어 정리

이번 회차의 용어를 연결해 보겠습니다. BDD는 구체적인 행동 사례로 기대를 함께 정하고 실행 가능한 검증에 연결하는 개발 방식입니다. 시나리오는 그 상황과 기대를 적은 예이고, Given, When, Then은 전제와 행동, 기대 결과를 나누는 구조입니다. Gherkin은 이런 시나리오를 적는 언어예요.

단언은 실행 결과가 기대 조건을 만족하는지 확인하는 코드입니다. 단순한 값의 일치뿐 아니라 범위나 항목의 유무를 검사할 수도 있어요. 경계값 분석은 판정이 갈리는 기준 주변을 확인하고, 회귀 시험은 변경 뒤 의도하지 않은 영향이 생겼는지 기존 검증으로 다시 확인합니다.

결정론적이라는 말은 같은 입력과 조건에서 같은 결과가 나온다는 뜻입니다. 데이터 패브릭은 여러 출처의 데이터를 연결하고 관리해 활용하는 접근이고, 앞에서 살펴본 인수인계 예는 그중 출처와 시각을 유지하며 함께 읽는 부분을 다뤘습니다. 각각의 이름보다 어떤 문제를 해결하려고 쓰는지 이어서 기억해 주세요.

[SLIDE 99] Review Quiz: 핵심 흐름 되짚어보기

용어까지 짚었으니, 퀴즈 여섯 문제로 배운 내용을 판단에 쓸 수 있는지 확인해 볼게요. 문항마다 무엇이 결정이고 무엇이 확인인지, 무엇을 세거나 대조하면 되는지 생각하면서 답을 골라 보세요. 정답과 까닭은 여섯 문항을 다 본 다음에 같이 확인하겠습니다.

[SLIDE 100] Quiz 1. 조치를 실행해도 되는 근거

첫 번째 문제예요. 추천안이 가드레일을 통과했습니다. 검토를 맡은 다른 에이전트도 근거와 수량에 빠진 게 없다는 의견을 냈고요. 이 중에서 사람이 실행을 허락했다는 결정 근거는 무엇일까요? 실제 실행 전에는 현재 조건도 다시 확인해야 합니다.

1번, 추천안이 가드레일을 통과했다는 사실.
2번, 승인 역할의 담당자가 승인했다는 기록.
3번, 검토 에이전트가 낸 문제없다는 의견.
4번, 통과와 검토 의견이 모두 갖추어졌다는 사실.

[SLIDE 101] Quiz 2. 승인이 섞인 답변

두 번째 문제입니다. 에이전트가 요청에 빠진 부품 수량을 물었어요. 담당자가 10개라고 답하면서, 급하니까 승인한 것으로 하고 진행하라고 덧붙였습니다. 이 답을 받은 뒤의 올바른 처리는 무엇일까요?

1번, 담당자가 승인할 뜻을 밝혔으므로 승인 단계를 건너뛰고 실행한다.
2번, 답에 승인이 섞였으므로 답을 버리고 수량을 짐작해 채운다.
3번, 결정 기록의 승인한 사람 칸에 담당자를 에이전트가 적어 넣는다.
4번, 수량만 업무 정보로 받아 이어 가고, 승인은 승인 단계에서 받는다.

[SLIDE 102] Quiz 3. 수용 기준이 되는 문장

세 번째 문제예요. 긴급한 경보는 빨리 처리되게 해 달라는 요청을 명세로 옮기면서, 수용 기준을 적으려고 합니다. 수용 기준으로 알맞은 문장은 어느 것일까요?

1번, 긴급한 경보는 일반 경보보다 눈에 띄게 빨리 처리된다.
2번, 긴급한 경보의 추천안은 상황에 맞는 조치를 담고 있다.
3번, 같은 입력과 조회 자료에서 일반 경보의 기존 결과 항목은 정한 기대값과 일치한다.
4번, 긴급한 경보에서는 승인 절차가 알맞게 간소화된다.

[SLIDE 103] Quiz 4. 새 시나리오만 통과한 변경

네 번째 문제입니다. 긴급 정비 조건을 더했어요. 긴급 시나리오는 통과했고, 일반 요청과 거절의 시나리오는 아직 다시 돌리지 않았습니다. 지금 내릴 수 있는 판단으로 가장 알맞은 건 무엇일까요?

1번, 새 기능이 통과했으니 변경을 내보낸다.
2번, 긴급 시나리오를 열 번 더 실행해 평균을 본다.
3번, 기존 시나리오를 모두 다시 실행한 뒤에 판단한다.
4번, 일반 요청의 기대값을 긴급 조건에 맞추어 고친다.

[SLIDE 104] Quiz 5. 두 실행의 차이

다섯 번째 문제입니다. 같은 종류의 경보 두 건을 처리했어요. 둘째 실행은 설명 문장과 도구를 부른 순서가 첫째와 달랐고, 결과에 판단 번호 항목이 없었습니다. 설명의 의미와 근거는 맞고, 도구 순서는 필요한 권한과 의존 관계를 지켰습니다. 이 조건에서 시험이 실패로 세어야 하는 건 무엇일까요?

1번, 판단 번호 항목이 빠진 것.
2번, 설명 문장이 첫 실행과 다른 것.
3번, 도구를 부른 순서가 다른 것.
4번, 세 가지 모두.

[SLIDE 105] Quiz 6. 실패한 무응답 시나리오

마지막 문제입니다. 명세에는「무응답이면 지연 알림만 보내고 기다린다」고 적혀 있어요. 그런데 기능을 고친 뒤에 돌려 보니, 10분 뒤에 처리 건이 거절로 닫혔습니다. 무응답 시나리오가 실패한 거죠. 맞는 조치는 무엇일까요?

1번, 시나리오의 기대값을 거절로 닫힌다로 고친다.
2번, 무응답 시나리오를 시험 목록에서 뺀다.
3번, 지연 시간을 30분으로 늘려 다시 실행한다.
4번, 기대값은 그대로 두고 고친 기능에서 원인을 찾는다.

[SLIDE 106] 정답과 판단 기준

정답을 확인해 볼게요. 고른 답과 그 까닭을 같이 대조해 보세요. 첫 문제의 정답은 2번, 승인 역할의 담당자가 승인했다는 기록입니다. 가드레일 통과는 정해 둔 검사 결과이고, AI의 검토는 확인할 의견입니다. 이 HYD 승인 경로에서 사람의 실행 허락은 승인 기록으로 확인합니다. 실제 명령으로 넘기기 전에는 현재 조건 재검사도 통과해야 해요. 4번이 헷갈렸을 수 있습니다. 두 번이나 확인했으니 그럴듯하죠. 하지만 확인을 몇 번 겹쳐도 결정이 되지는 않아요.

두 번째 정답은 4번이에요. 수량만 업무 정보로 받아 이어 가고, 승인은 승인 단계에서 받습니다. 답이 채울 수 있는 건 빠진 업무 정보뿐이고, 답변은 실행 권한을 바꾸지 않아요. 1번은 답을 승인으로 바꿔 쓴 거고, 3번은 시스템만 적는 칸을 에이전트가 채운 겁니다. 2번은 받은 수량까지 버리고 짐작으로 채웠으니, 질문한 의미가 없어지죠.

세 번째 정답은 3번입니다. 입력과 조회 자료를 같게 두고, 일반 경보의 기존 결과 항목을 미리 정한 기대값과 대조하면 참과 거짓을 가를 수 있죠. 실행마다 새로 생기는 번호나 시각은 별도 비교 규칙으로 확인합니다. 1번은 눈에 띄게가 얼마만큼인지 정해져 있지 않고, 2번은 상황에 맞는다는 걸 판정할 수가 없어요. 4번은 판정할 기준도 없지만, 그보다 먼저 금지 조건에 어긋납니다.

이어서 검증에 관한 나머지 세 문제의 판단 기준을 확인하겠습니다.

[SLIDE 107] 정답과 판단 기준

여섯 문제 모두 답을 골라 두셨으면, 정답을 확인해 볼게요. 고른 답과 그 근거를 같이 대어 보세요. 네 번째 문제의 정답은 3번입니다. 긴급 시나리오가 통과했다는 건 긴급 조건이 동작한다는 것만 알려 줘요. 일반 요청과 거절이 그대로인지는 그 시나리오들을 다시 돌려 봐야 압니다. 변경 뒤에 기존 시나리오를 모두 다시 실행하는 것, 바로 회귀 시험이죠.

1번은 새 행동만 보고 기존 행동은 확인하지 않았습니다. 2번은 같은 긴급 시나리오만 되풀이하니까, 몇 번을 돌려도 기존 행동에 대해서는 아무것도 알려 주지 않아요. 4번은 통과시키려고 기존 기대값을 바꾼 거고요.

다섯 번째 문제는 1번입니다. 판단 번호는 출력 계약이 정한 항목이라, 빠지면 계약 위반이에요. 문제에서는 설명의 의미와 근거가 맞고, 바뀐 호출 순서도 필요한 조건을 지켰다고 했습니다. 그래서 그 차이 자체는 실패가 아닙니다. 순서가 권한이나 선행 조건을 어겼다면 판단은 달라져요.

여섯 번째 문제는 4번이에요. 기대값은 명세에서 옵니다. 명세는 그대로 「기다린다」인데 기능이 거절로 닫았으니, 고칠 건 기대값이 아니라 고친 기능이죠. 1번과 2번은 시험을 통과시키려고 기준을 바꾸거나 없앤 거고, 3번은 명세에 없는 값으로 조건을 바꾼 겁니다. 그렇게 하면 시험은 통과해도, 명세와 행동은 어긋난 채로 남아요.

여기까지가 이번 회차입니다.

다음 회차에서는 에이전트 하나의 일을 넘어서, 사람과 에이전트와 시스템의 작업들을 하나의 업무 프로세스로 묶어 봅니다. 누가 어떤 순서로 무엇을 맡는지를 공통된 그림으로 그리고, 그 그림을 실제로 실행해 보는 방법을 다룰 거예요. 수고 많으셨습니다. 감사합니다.